

# PROTECTED TAILS AND POLYNOMIAL-TIME ENUMERATION OF PERMUTATIONS AVOIDING A DIRECT SUM OF AN INCREASING PATTERN AND 231

HENNING ARNÓR SKEGGI ÚLFARSSON

**ABSTRACT.** We give an algorithm counting the permutations that avoid a fixed pattern of the following form: the direct sum of an increasing pattern and 231. The first members of the family are 1342 and 12453. For each member the algorithm uses polynomially many operations and stored integers, with degrees that grow linearly in the length of the pattern. It comes from a recurrence that reads a permutation from left to right and records the constraints that the letters read so far impose on those still unread. This recurrence has exponentially many states, but part of each state is protected: later steps carry it along unchanged and do not depend on it, and factoring the protected part out leaves a dynamic program of polynomial size. For 12453 a translation symmetry sharpens the bounds to degree seven for the operations and degree four for the storage. We also compute the number of 12453-avoiding permutations of every length up to 150. The previously published series, due to Biers-Ariel (2019), reached length 38. We also give a sampler of uniformly random avoiders. A floating-point implementation of it, proved to be within total variation distance  $3.5 \cdot 10^{-5}$  of uniform for ideal random bits, draws the one million 12453-avoiding permutations of length 300 shown in a heatmap. The literal and kernel recurrences for 1342 and 12453 are verified in the Lean 4 proof assistant.

## 1. INTRODUCTION

A permutation  $\pi = \pi_1 \cdots \pi_n$  of  $[n] = \{1, \dots, n\}$  *contains* another permutation  $\tau$  of  $[k]$  if there are indices  $1 \leq i_1 < \cdots < i_k \leq n$  for which  $\pi_{i_1}, \dots, \pi_{i_k}$  has the same relative order as  $\tau$ . In this case we say that  $\tau$  is a *pattern* in  $\pi$ . Otherwise  $\pi$  *avoids*  $\tau$ . For example, the entries 2, 5, 7, 9, 6 in 251796834 have relative ranks 1, 2, 4, 5, 3, and therefore form an occurrence of 12453. We write  $\text{Av}_n(\tau)$  for the set of  $\tau$ -avoiding permutations of  $[n]$ , and  $\text{Av}(\tau) = \bigcup_{n \geq 0} \text{Av}_n(\tau)$ . Throughout,  $\mathbb{N} = \{0, 1, 2, \dots\}$ .

The basic enumerative problem is to determine

$$a_n(\tau) = |\text{Av}_n(\tau)|.$$

By *polynomial-time enumeration* we mean computing this integer, not listing the avoiding permutations. In other words, we seek a *Wilfian formula* in the sense of Wilf [25] and Zeilberger [26]. For a fixed pattern, the theorem of Marcus and Tardos [19] gives an exponential upper bound  $a_n(\tau) \leq c(\tau)^n$  on the counting sequence. Arratia [6] showed that  $a_n(\tau)^{1/n}$  has a limit in  $[1, \infty]$ . The limit, which is finite by this bound, is called the *Stanley–Wilf limit* of  $\tau$ , or the *growth rate* of  $\text{Av}(\tau)$ . The task is to obtain  $a_n(\tau)$  in at most  $P(n)$  steps, where  $P$  is a polynomial. No general principle can apply to all finitely based classes, since Garrabrant and Pak [16] constructed one whose counting sequence cannot be computed in polynomial time unless  $\text{EXP} = \oplus \text{EXP}$ .<sup>1</sup>

---

*Date:* September 2026.

*2020 Mathematics Subject Classification.* 05A05, 05A15, 05A16, 68W40, 68V15.

*Key words and phrases.* permutation pattern, pattern avoidance, polynomial-time algorithm, Wilfian formula.

<sup>1</sup>Here  $\text{EXP}$  is the class of decision problems solvable in deterministic exponential time,  $\oplus \text{EXP}$  is the class of problems that ask whether a nondeterministic exponential-time machine has an odd number of accepting computations, and the two classes are widely believed to differ.

If  $\alpha \in S_r$  and  $\beta \in S_s$ , their *direct sum*  $\alpha \oplus \beta$  is the word obtained by writing  $\alpha$  and then writing  $\beta$  with every value increased by  $r$ . We will focus on the following direct-sum family. Put

$$\iota_d = 12 \cdots d, \quad \beta_d = \iota_d \oplus 231 = 12 \cdots d(d+2)(d+3)(d+1). \quad (1)$$

Thus

$$\beta_1 = 1342, \quad \beta_2 = 12453, \quad \beta_3 = 123564, \quad \beta_4 = 1234675.$$

The case  $d = 1$  is already enumerated algebraically by Bóna [10]. The family itself also appears in Bóna [11]: from  $\beta_d = 1 \oplus \beta_{d-1}$  for  $d \geq 2$  he obtained a recursive deletion rule for *left-to-right minima* (entries smaller than every entry to their left), and, starting from the Stanley–Wilf limit 8 of 1342 [10], used it to determine the Stanley–Wilf limits of the family [11, Proposition 5.1 and Theorem 5.2]. The rule determines the growth rates but not the exact counts. The next case is  $d = 2$ , giving  $\beta_2 = 12453$ . The counting sequence begins

$$1, 1, 2, 6, 24, 119, 694, 4581, 33286, 260927, 2174398, 19053058, \dots \quad (2)$$

In the census of Clisby, Conway, Guttmann, and Inoue [13], the 12453 sequence was known through  $n = 38$  from a specialized program of Biers-Ariel [8], posted with OEIS A116485 [21]. The program uses a composition-valued state of unbounded length, and the number of its states grows exponentially. Our starting point is the same recurrence: for  $d = 2$  the literal recurrence developed below coincides with Biers-Ariel’s, state for state (Section 4), and we extend it to the whole family  $\beta_d$ . We then show how to shrink its exponential state space to polynomial size.

The family also has a description by a sorting device. Albert, Homberger, Pantone, Shar, and Vatter [3] study the  $\mathcal{C}$ -machine, which reads the entries  $1, 2, \dots, n$  in order and either outputs each one directly or pushes it anywhere into a container whose contents must stay order-isomorphic to a permutation in  $\mathcal{C}$ , and which outputs entries from the container only from its left end. Their Theorem 1.1 shows that the  $\text{Av}(B)$ -machine generates exactly  $\text{Av}(1 \ominus B)$ , and they observe that  $\text{Av}(4213)$ , the complement of  $\text{Av}(1342)$ , is generated by the  $\text{Av}(213)$ -machine, answering a question of Bóna. Since  $\beta_d^c = 1 \ominus \beta_{d-1}^c$  for the complement  $\beta_d^c$ , the class  $\text{Av}(\beta_d)$  is, up to complementation, the class generated by the  $\text{Av}(\beta_{d-1}^c)$ -machine. For 12453 the container class is  $\text{Av}(4213)$ . They prove that the generated class has a rational generating function when  $\mathcal{C}$  is finite, an algebraic one when  $\mathcal{C}$  is bounded, that is, when  $|\mathcal{C}_n|$  is bounded by a constant, and a polynomial-time counting algorithm when  $|\mathcal{C}_n|$  is bounded by a polynomial in  $n$ . The container class  $\text{Av}(4213)$  has growth rate 8, so none of these conditions applies to 12453.

Suppose  $\pi$  avoids  $\beta_d$ . Every occurrence of  $12 \cdots d$  in a prefix of  $\pi$  forces the unread values above it to be read in a 231-avoiding order, and we record the conditions created by a prefix as an ordered stack of intervals of unread values (Section 2). Reading one more letter changes the control, that is, the counts of unread values that no condition restricts, or the active head of the stack, or both. The deferred intervals are not touched until the head has been consumed, and the transitions up to that point do not depend on them. A transfer kernel records the control at which this happens. This removes the stack from the memoization key, and the resulting table has polynomial size.

Our main result is the following. The notation  $O_d(\cdot)$  allows the hidden constant to depend on the fixed family parameter  $d$ .

**Theorem 1.1.** *For every fixed  $d \geq 1$  and every  $N \geq 1$ , the numbers*

$$|\text{Av}_0(\beta_d)|, \dots, |\text{Av}_N(\beta_d)|$$

*can be computed using  $O_d(N^{3d+2})$  integer arithmetic operations and  $O_d(N^{2d+1})$  stored integers. All integers involved have  $O_d(N \log N)$  bits. Therefore the bit complexity<sup>2</sup> is polynomial. More precisely it is*

$$O_d\left(N^{3d+2}M(N \log N)\right),$$

*where  $M(b)$  is the cost of multiplying  $b$ -bit integers, and the bit storage is  $O_d(N^{2d+2} \log N)$ .*

In particular, the family theorem gives  $O(N^5)$  operations and  $O(N^3)$  stored integers for  $\text{Av}(1342)$ , and  $O(N^8)$  and  $O(N^5)$  for  $\text{Av}(12453)$ . An additional translation symmetry, proved in [Section 7](#), improves this to  $O(N^7)$  and  $O(N^4)$  for  $\text{Av}(12453)$  ([Corollary 7.4](#)). The same symmetry holds for every  $d$  and lowers both exponents of [Theorem 1.1](#) by one. The theorem gives a polynomial bound for each fixed  $d$ , not a uniform one when  $d$  is part of the input.

To our knowledge, no polynomial-time enumeration algorithm was previously available for the family  $\text{Av}(\iota_d \oplus 231)$  at every fixed  $d$ , or even for the single class  $\text{Av}(12453)$ . In this paper we show that the literal recurrence is a pushdown system: every transition acts on the interval stack through its active head alone, so the completion counts factor through transfer kernels ([Theorem 2.14](#) and [Corollary 5.1](#)), although the number of reachable stack states is at least the Fibonacci number  $F_{n-d}$  ([Proposition 4.5](#)). The factorization itself is the counting form of the classical conversion of a pushdown automaton into a grammar ([Remark 2.15](#)). What is new is the combinatorial input that makes it apply: for every  $d$ , the conditions created by a prefix form a stack that the scan touches only at its head ([Lemma 2.7](#) and [Propositions 2.8](#) and [4.1](#)), and the number of controls is polynomial in  $n$ , so the factorization gives a transfer table of polynomial size rather than a finite grammar. We prove the family bounds of [Theorem 1.1](#) and two translation quotients for 12453 ([Lemmas 7.2](#) and [7.3](#)). We also provide the values  $|\text{Av}_n(12453)|$  for  $n \leq 150$ , computed by Chinese remaindering with a proved bound and checked by separately written implementations ([Table 1](#) and the supplement). The transfer tables can also be used to generate uniformly random elements of  $\text{Av}_n(12453)$  in polynomial time ([Proposition 8.1](#)), and the literal and kernel recurrences for  $d \leq 2$  are machine-checked in Lean 4 ([Section 9](#)).

The paper is organized as follows. In [Section 2](#) we describe the conditions that a scanned prefix imposes on its completions, represent them by an interval stack, and develop the whole method for  $d = 1$ , where  $\text{Av}(1342)$  serves as a prototype: the literal recurrence, the protected-tail factorization of [Theorem 2.14](#), and the resulting polynomial algorithm. In [Section 3](#) we replace the single minimum by the  $d$  patience-sorting thresholds and their bands. In [Section 4](#) we derive the literal recurrence for every  $d$  and show that its state space is exponential. In [Section 5](#) we apply the factorization to it and obtain the transfer kernels. In [Section 6](#) we bound their supports and prove [Theorem 1.1](#). In [Section 7](#) we specialize to 12453 and use a translation symmetry to reach  $O(N^7)$  operations and  $O(N^4)$  storage, and a second one to halve the storage constant. In [Section 8](#) we compute the values  $|\text{Av}_n(12453)|$  for  $n \leq 150$  and describe the uniform sampler. In [Section 9](#) we describe the Lean development, with details in [Appendix B](#). In [Section 10](#) we place the class among the length-five Wilf classes and state a conjecture on its asymptotics. [Appendix A](#) bounds the distance of the floating-point sampler from uniformity.

## 2. TRIGGERS, STACKS, AND THE COMPLETE ONE-THRESHOLD PROTOTYPE

We first describe the restrictions that a scanned prefix imposes on its completions and show that they admit an interval-stack representation for every  $d \geq 1$ . We then develop the counting

---

<sup>2</sup>The operation count treats each integer addition or multiplication as one step. The bit complexity counts operations on single bits, so the sizes of the integers enter through the multiplication cost  $M(b)$  defined below.

algorithm completely for  $d = 1$ , which isolates the protected-tail construction before the additional threshold coordinates are introduced. The scan states and their legal moves (Definition 2.4, Lemmas 2.5 and 2.7, and Proposition 2.8) are defined and studied for every  $d$  and used unchanged in Section 4, and the proof of Theorem 2.14 applies verbatim in Section 5.

Containment and avoidance are defined for words with distinct entries in the same way, by relative order. The *standardization* of such a word is obtained by replacing the smallest entry by 1, the next smallest by 2, and so on. If  $X$  is a set of values,  $w|_X$  denotes the subword formed by retaining only the entries belonging to  $X$ . An *increasing  $d$ -subsequence* of a permutation  $\pi$ , or more generally of a word  $\pi$  with distinct entries, is a choice of positions  $i_1 < \dots < i_d$  with  $\pi_{i_1} < \dots < \pi_{i_d}$ . We call its last entry a  *$d$ -trigger*. The positions need not be adjacent.

**Lemma 2.1** (Trigger lemma). *A permutation  $\pi = \pi_1 \dots \pi_n$  avoids  $\beta_d$  if and only if, for every increasing  $d$ -subsequence ending at  $c = \pi_j$ , the word*

$$\pi_{j+1} \dots \pi_n|_{\{x: x > \pi_j\}}$$

*avoids 231.*

*Proof.* Suppose  $\pi$  contains  $\beta_d$ . The first  $d$  entries of an occurrence form an increasing  $d$ -subsequence, and its last three entries form a 231 whose values are all larger. Conversely, an increasing  $d$ -subsequence followed by such a 231 has relative order  $\iota_d \oplus 231 = \beta_d$ .  $\square$

When a trigger  $c$  is read, its *projection*, the subword of the letters after  $c$  that are larger than  $c$ , must avoid 231. Later in the scan, some letters of that projection have already been read. It is not enough to test the unread part of the projection, since an occurrence of 231 may use both read and unread letters. The interval stack below records the resulting conditions on the unread letters.

More formally, the 231-obligation created by a trigger  $c = \pi_j$  requires that  $\pi_{j+1} \dots \pi_n|_{\{x: x > c\}}$  avoid 231. Let  $\sigma = \sigma_1 \dots \sigma_k$  be the scanned prefix. The letters of  $[n]$  not in  $\sigma$  are *unread*, and a *completion* of  $\sigma$  is a word  $w$  using each unread letter once.

**2.1. The Catalan interval stack.** Here and throughout, an *interval* is a consecutive block of the ordered set of currently unread values. Its elements need not be consecutive integers. The *local rank* of an element  $x$  of a finite totally ordered set  $I$  is  $|\{y \in I : y \leq x\}|$ . The following first-letter decomposition of  $\text{Av}(231)$  applies to an arbitrary finite ordered alphabet.

**Lemma 2.2** (First-letter lemma). *Let  $I$  be a finite totally ordered set of size  $\ell$ , and let  $x \in I$  have local rank  $r$ . A permutation of  $I$  beginning with  $x$  avoids 231 if and only if all values below  $x$  occur before all values above  $x$ , and the two induced subwords avoid 231. Thus reading  $x$  replaces an interval of size  $\ell$  by intervals of sizes  $r - 1$  and  $\ell - r$ , in that order, with zero parts deleted.*

*Proof.* Suppose an upper value occurs before a lower value after  $x$ . Then  $x$  and these two values form a 231. Conversely, if the lower and upper subwords occur in that order, a 231 cannot use letters from both: both position and value increase from the first block to the second. Nor can a 231 use  $x$  and two letters from just one block:  $x$  is larger than every lower letter and smaller than every upper letter, while the first entry of a 231 has middle rank. If it uses  $x$ , one lower letter, and one upper letter, their order is 213. Avoidance therefore reduces to avoidance inside the two blocks.  $\square$

We write  $\text{Av}(231)(I)$  for the 231-avoiding words using each value of  $I$  once. If  $I < J$ , that is, every element of  $I$  is smaller than every element of  $J$ , and  $u$  and  $w$  are words on  $I$  and  $J$ , then  $u \oplus w$  is the concatenation  $uw$ . For sets of such words,  $\oplus$  denotes the set of all concatenations. Consider the projection created by one trigger  $c$ , and read its letters one at a time. By Lemma 2.2, the first letter  $x$  splits the rest of the projection into a lower and an upper interval, and every

letter of the lower interval must be read before any letter of the upper one. The next letter of the projection therefore lies in the lower interval while that interval is nonempty, and the lemma, applied to the subword on that interval, splits it in turn. The letter itself then plays no further role, since both conditions of the lemma concern only the letters after it. Repeated application of [Lemma 2.2](#) thus represents what the obligation of  $c$  requires of the unread letters by an ordered interval stack

$$I_1 < I_2 < \cdots < I_s,$$

which we also write  $I_1 \mid I_2 \mid \cdots \mid I_s$ . That is, the unread letters of the projection must form a word in

$$\text{Av}(231)(I_1) \oplus \text{Av}(231)(I_2) \oplus \cdots \oplus \text{Av}(231)(I_s). \quad (3)$$

The read letters of the projection do not appear in (3), and none of them is needed again. For example, relabel the values of a projection as  $1, \dots, 6$  and suppose that its first two letters are 4 and 2. Reading 4 leaves the stack  $\{1, 2, 3\} \mid \{5, 6\}$ , and reading 2 then leaves  $\{1\} \mid \{3\} \mid \{5, 6\}$ . The completion 1365 is allowed, while a completion beginning 31 is excluded: together with the read letter 2 it would form a 231.

The stack records the obligations created by the scanned prefix. Further obligations arise as the completion is read, and they are incorporated by the stack updates described below. We call  $I_1$  the *active head* and  $(I_2, \dots, I_s)$  the *deferred tail*. Only the active head may be read next among the values of the stack: every letter of  $I_1$  must be read before any letter of  $I_2$ , and so forth. Unread values below the trigger are constrained by no obligation created so far and may be interspersed. For an interval of size  $\ell$ , an *endpoint choice* has local rank 1 or  $\ell$ , and an *interior choice* has local rank in  $\{2, \dots, \ell - 1\}$ .

We call the gap in value between two consecutive intervals of the current stack a *separation*, and a read letter lying in that gap a *separator*. A separation may have several separators. An interior choice creates a new separation, at the letter read, while an endpoint choice leaves at least one side empty and creates none, although the letter read may lie in an existing separation. [Lemma 2.7](#) below determines the separations of the stack once the obligations of several triggers have been combined.

**Example 2.3** (Reading one projection). The permutation

$$\pi = 9, 11, 10, 14, 5, 12, 6, 2, 3, 8, 4, 7, 1, 13, 15 \quad (4)$$

of [15] avoids 1342, and therefore also 12453, and serves as the running example of this paper. For  $d = 1$  every letter is a trigger. The projection created by the first letter 9 is the word 11, 10, 14, 12, 13, 15 formed by the later letters above 9. It standardizes to 215346 and avoids 231, as [Lemma 2.1](#) requires. We read this projection letter by letter. The stack starts as the single interval  $\{10, \dots, 15\}$ . The first letter 11 has local rank 2, an interior choice, so [Lemma 2.2](#) requires the lower interval  $\{10\}$  to be read before the upper interval  $\{12, \dots, 15\}$ . The stack is  $\{10\} \mid \{12, \dots, 15\}$ . Reading 10 is an endpoint choice that exhausts the head and leaves  $\{12, \dots, 15\}$ , and the interior choice 14, of local rank 3 in that interval, splits it into  $\{12, 13\} \mid \{15\}$ . The unread letters of the projection must now form a word in  $\text{Av}(231)(\{12, 13\}) \oplus \text{Av}(231)(\{15\})$ : either order of 12 and 13, then 15. The read letters 11, 10, 14 are not retained. While 10 was unread, 11 was the only separator of the separation between  $\{10\}$  and  $\{12, \dots, 15\}$ . Now 14 is the only separator of the separation between  $\{12, 13\}$  and  $\{15\}$ , and 10 and 11 separate nothing.

Several triggers impose overlapping obligations. We now define the object that records all of them: a read prefix together with a stack, updated by three kinds of move. [Proposition 2.8](#) then shows that the legal prefixes are exactly the prefixes of  $\beta_d$ -avoiding permutations.

**Definition 2.4** (Scan states and legal moves). Fix  $d \geq 1$  and  $n$ . A *scan state* is a pair  $(\sigma, S)$ . Here  $\sigma$  is a word with distinct letters from  $[n]$ , called the *read prefix*, and the letters of  $[n]$  not in

$\sigma$  are *unread*. We write  $q$  for the least  $d$ -trigger of  $\sigma$ , and put  $q = n + d$ , a virtual value above every letter, when  $\sigma$  has no  $d$ -trigger. The *stack*  $S = (I_1, \dots, I_s)$  is a list of nonempty sets with  $I_1 < I_2 < \dots < I_s$  whose union is the set of unread letters larger than  $q$ , so that each  $I_j$  is an interval in the sense of [Section 2.1](#),  $I_1$  is the active head, and  $(I_2, \dots, I_s)$  is the deferred tail. In particular  $S = \emptyset$  is the empty list,  $s = 0$ , when  $\sigma$  has no  $d$ -trigger. The *initial state* is  $(\varepsilon, \emptyset)$ : the empty prefix and the empty stack. Let  $(\sigma, S)$  be a scan state and let  $x$  be an unread letter. Reading  $x$  is a *legal move*, leading to the state  $(\sigma x, S')$ , in the following three cases.

- (a) *Merger*:  $x$  is a  $d$ -trigger of  $\sigma x$  and  $x < q$ . Let  $E$  be the set of unread letters strictly between  $x$  and  $q$ . Then  $S' = (E \cup I_1, I_2, \dots, I_s)$ , where  $I_1 = \emptyset$  if  $s = 0$ , with an empty set deleted.
- (b) *Split*:  $x$  is a  $d$ -trigger of  $\sigma x$ ,  $x > q$ , and  $x$  lies in the active head  $I_1$ . Then  $S' = (I_1^{<x}, I_1^{>x}, I_2, \dots, I_s)$ , where  $I_1^{<x}$  and  $I_1^{>x}$  are the sets of elements of  $I_1$  below and above  $x$ , with empty sets deleted. The move is an endpoint or an interior choice, as defined above, depending on the local rank of  $x$  in  $I_1$ .
- (c) *Non-trigger move*:  $x$  is not a  $d$ -trigger of  $\sigma x$ . Then  $S' = S$ . For  $d = 1$  every letter is a 1-trigger, so this case does not occur.

Reading any other unread letter is *illegal*. A word  $\sigma$  is *legal* if the initial state reaches a state  $(\sigma, S)$  by legal moves, and we write  $\mathcal{S}(\sigma)$  for the resulting stack, the *stack of  $\sigma$* .

A  $d$ -trigger read from a prefix without one always merges, since then  $q = n + d$ . Each move determines  $S'$  from  $(\sigma, S)$  and  $x$ , and [Lemma 2.5\(a\)](#) below shows that  $(\sigma x, S')$  is again a scan state, so the stack  $\mathcal{S}(\sigma)$  of a legal word is determined by  $\sigma$ . Note that every unread letter  $x > q$  is a  $d$ -trigger of  $\sigma x$ , since replacing the last entry of an increasing  $d$ -subsequence ending at  $q$  by  $x$  gives one ending at  $x$ . For  $d = 1$  a legal move is a new left-to-right minimum, which merges ([Figure 1\(a\)](#)), or a letter of the active head, which splits ([Figure 1\(b\)](#)). [Figure 2](#) shows the merger of the running example in detail.

**Lemma 2.5** (Legal moves). *Let  $(\sigma, S)$  be a scan state and let  $x$  be an unread letter.*

- (a) *If reading  $x$  is a legal move, then  $(\sigma x, S')$ , with  $S'$  as in [Definition 2.4](#), is a scan state.*
- (b) *Reading  $x$  is illegal if and only if  $x$  lies in one of the deferred intervals  $I_2, \dots, I_s$ .*

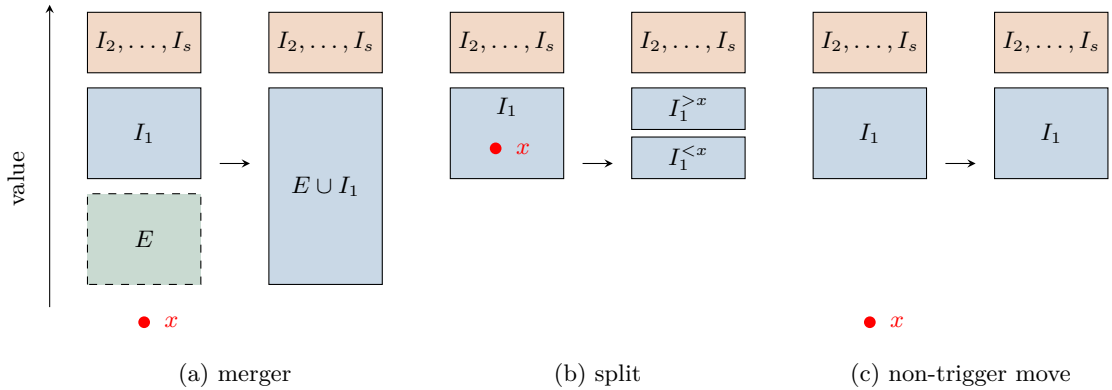


FIGURE 1. The three legal moves of [Definition 2.4](#), with values increasing upwards and the active head  $I_1$  at the bottom of the stack. In a merger the letter  $x$  lies below the least  $d$ -trigger  $q$ , and the unread values  $E$  strictly between  $x$  and  $q$  join the head. In a split  $x$  lies in the head and cuts it into the values below and above  $x$ . A non-trigger move leaves the stack unchanged. The deferred tail is unchanged in all three moves.

*Proof.* (a) In a merger,  $x$  becomes the least  $d$ -trigger of  $\sigma x$ , and the unread letters of  $\sigma x$  larger than  $x$  are those of  $E$  together with those larger than  $q$ , that is,  $E \cup I_1 \cup \dots \cup I_s$ , the union of  $S'$ . Every letter of  $E$  is smaller than  $q$  and every letter of  $S$  is larger, so  $E < I_1$  and the sets of  $S'$  are disjoint and increasing. In a split, the least  $d$ -trigger is still  $q$ , since  $x > q$ , and the unread letters above  $q$  lose exactly  $x$ , which is what the split removes. The two parts of  $I_1$  lie below and above  $x$ . In a non-trigger move,  $x < q$ , since an unread  $x > q$  is a  $d$ -trigger of  $\sigma x$  as noted above, so  $q$  and the unread letters above it are unchanged, and so is  $S$ .

(b) Let  $x$  lie in a deferred interval. Then  $x > q$ , so  $x$  is a  $d$ -trigger of  $\sigma x$ , as noted above. Therefore cases (a) and (c) of [Definition 2.4](#) do not apply, and  $x \notin I_1$  excludes case (b). Conversely, let reading  $x$  be illegal. Since case (c) does not apply,  $x$  is a  $d$ -trigger of  $\sigma x$ , and since case (a) does not apply,  $x > q$ . Therefore  $x$  lies in the stack, and in a deferred interval since case (b) does not apply.  $\square$

**Example 2.6** (One merger). We continue [Example 2.3](#). The fifth letter 5 of  $\pi$  is a trigger smaller than the earlier triggers 9, 11, 10, 14, so reading it is a merger, and [Figure 2](#) shows the stack before and after. In the notation of [Definition 2.4\(a\)](#),  $q = 9$  and  $E = \{6, 7, 8\}$ , the unread values between 5 and 9. The stack  $\{12, 13\} \mid \{15\}$  consists of the unread values above 9, and the merged stack is  $\{6, 7, 8, 12, 13\} \mid \{15\}$ . The values 6, 7, 8 join the head without a separation: the read letters 9, 10, 11, which lie between them and 12 in value, were read before the trigger 5 and constrain nothing in  $E$ . They must, however, precede 15: if 15 came before some  $y' \in \{6, 7, 8\}$ , then either 15 precedes some  $y \in \{12, 13\}$  and 9, 14, 15,  $y$  is an occurrence of 1342, or 12 precedes 15 and 5, 12, 15,  $y'$  is one. Inside the head only 231-avoidance remains. The permutation  $\pi$  continues with 12 before 6, and 13, 12, 6, 7, 8 would have been allowed as well. For  $d = 2$  the same permutation is scanned in [Example 4.2](#): there the first 2-trigger is 11 rather than 9, and the corresponding merger occurs at the 2-trigger 6, with  $q = 10$  and  $E = \{7, 8\}$ .

Two unread letters are *adjacent* if no unread letter lies strictly between them. The following lemma determines the separations of the stack from the scanned prefix.

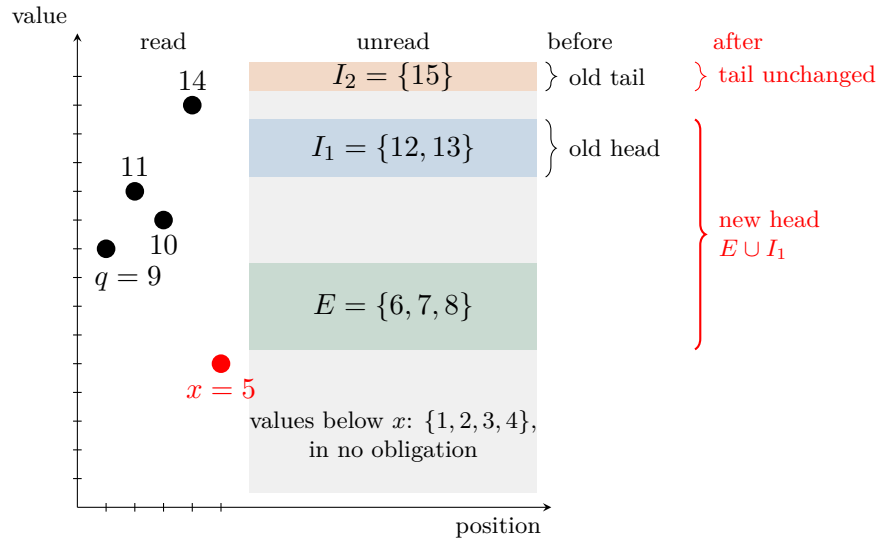


FIGURE 2. A merger ([Definition 2.4\(a\)](#)) for  $d = 1$  on the running example (4): the prefix 9, 11, 10, 14 has been read, leaving the head  $I_1 = \{12, 13\}$  and the tail  $I_2 = \{15\}$ , and the new trigger  $x = 5$  merges  $E = \{6, 7, 8\}$ , the unread values between  $x$  and  $q = 9$ , into the head. The tail is unchanged. See [Example 2.6](#).

**Lemma 2.7** (Separations of the stack). *Let  $\sigma$  be a legal word, let  $q$  be its least  $d$ -trigger, and let  $u < v$  be adjacent unread letters above  $q$ . Then  $u$  and  $v$  lie in different intervals of  $\mathcal{S}(\sigma)$  if and only if some letter  $x$  with  $u < x < v$  was read after a  $d$ -trigger smaller than  $u$ .*

*Proof.* We argue by induction along the legal moves that read  $\sigma$ . While no  $d$ -trigger has been read there is no letter above  $q = n + d$ . Let  $x$  be the letter read next. Reading  $x$  makes its two unread neighbours adjacent, when both exist, and leaves every other adjacent pair with its members and the letters between them.

Suppose first that reading  $x$  is a merger. Then  $x$  becomes the least  $d$ -trigger, and an adjacent pair  $u < v$  above  $x$  that was not an adjacent pair above  $q$  has  $u \in E$ . Every  $d$ -trigger of  $\sigma$  is larger than  $u$ , and no letter has been read after  $x$ , so the second condition fails. The stack of  $\sigma$  consists of the unread letters above  $q$ , so  $v$  lies in  $E$  or is the least element of  $I_1$ , and  $u$  and  $v$  lie in the single interval  $E \cup I_1$ . Thus the first condition fails as well. The other pairs keep their intervals, and  $x$  lies below them and has not been followed by any letter, so both conditions are unchanged for them.

Suppose next that reading  $x$  is a split. Then  $x \in I_1$  and  $x > q$ . A new adjacent pair consists of the unread neighbours  $u < x < v$  of  $x$ , when both exist and  $u > q$ . Then  $u$  lies in the part of  $I_1$  below  $x$ , while  $v$  lies in the part above  $x$  or, if  $x = \max I_1$ , is the least element of  $I_2$ . Thus  $u$  and  $v$  lie in different intervals, and  $x$ , read after  $q < u$ , witnesses the second condition. The other pairs keep their intervals, and  $x$  lies between the members of none of them, since those are adjacent and  $x$  was unread.

Finally, a non-trigger move reads a letter  $x < q$ , as in the proof of Lemma 2.5, and changes neither the stack nor either condition.  $\square$

**Proposition 2.8** (Scan states of avoiders). *Let  $\sigma$  be a legal word with stack  $\mathcal{S}(\sigma) = (I_1, \dots, I_s)$ .*

- (a) *The word  $\sigma$  avoids  $\beta_d$ .*
- (b) *If reading the unread letter  $x$  from  $\sigma$  is illegal, then no  $\beta_d$ -avoiding permutation has the prefix  $\sigma x$ .*
- (c) *A word is legal if and only if it is a prefix of a  $\beta_d$ -avoiding permutation. In particular  $\text{Av}_n(\beta_d)$  is the set of legal permutations of  $[n]$ .*

*Proof.* (a) Suppose that  $\sigma$  contains  $\beta_d$ . The first  $d$  letters of an occurrence form an increasing  $d$ -subsequence ending at a  $d$ -trigger  $c$ , and its last three letters  $y_2, y_3, y_1$ , in this order, satisfy  $c < y_1 < y_2 < y_3$ . Let  $\sigma'$  be the prefix of  $\sigma$  read just before  $y_3$ . It is legal and contains  $c$  and  $y_2$ , so its least  $d$ -trigger  $q$  satisfies  $q \leq c$ . Since  $y_3 > q$  is read legally from  $\sigma'$ , it lies in the active head of  $\mathcal{S}(\sigma')$  by Lemma 2.5(b). Let  $u$  and  $v$  be the unread letters of  $\sigma'$  next below and next above  $y_2$ . Then  $q < y_1 \leq u$  and  $v \leq y_3$ , and  $y_2$  was read after the  $d$ -trigger  $c < u$ , so  $u$  and  $v$  lie in different intervals of  $\mathcal{S}(\sigma')$  by Lemma 2.7. But the stack consists of the unread letters above  $q$ , its active head is the lowest interval, and  $u < v \leq y_3$ , so  $u$  and  $v$  both lie in the active head, a contradiction.

(b) By Lemma 2.5(b),  $x$  lies in a deferred interval, so  $s \geq 2$ . Let  $u = \max I_1$  and  $v = \min I_2$ , which are adjacent. By Lemma 2.7, some letter  $z$  with  $u < z < v$  was read after a  $d$ -trigger  $c < u$ . A permutation with prefix  $\sigma x$  reads  $u$  after  $x$ , and  $v \leq x$ . An increasing  $d$ -subsequence ending at  $c$ , followed by  $z, x, u$ , is therefore an occurrence of  $\beta_d$ , since  $c < u < z < x$ .

(c) Every prefix of an avoider is legal: the empty prefix is, and if a prefix  $\sigma'$  of an avoider  $\pi$  is legal and the next letter  $x$  of  $\pi$  were illegal, (b) would forbid the prefix  $\sigma'x$  of  $\pi$ . Conversely, a legal  $\sigma$  has an avoiding completion: read the unread letters smaller than  $q$  in decreasing order, and then the intervals  $I_1, \dots, I_s$  in order, each in decreasing order. Each letter of the first group is the largest unread letter below  $q$ , so it is a non-trigger move or a merger with  $E = \emptyset$ , and the stack is unchanged. Each later letter is the greatest element of the active head, and therefore a  $d$ -trigger above  $q$ , so reading it is an endpoint choice. All moves are legal, so the completed

permutation is legal and avoids  $\beta_d$  by (a). The last sentence of (c) follows, with (a) for the legal permutations.  $\square$

Being a prefix of a  $\beta_d$ -avoiding permutation is stronger than avoiding  $\beta_d$ . The word 134 avoids 1342, but its only completion in [4] is 1342, and after 1, 3 the stack is  $\{2\} \mid \{4\}$ , so reading 4 is illegal.

**2.2. The one-threshold state.** For  $d = 1$ , every letter is a trigger, so the least  $d$ -trigger  $q$  of Definition 2.4 is the least letter  $m$  read so far, virtual for the empty prefix. A new left-to-right minimum joins the unread values between it and  $m$  to the active head, and a letter of the active head shortens the head at an endpoint choice and splits it at an interior choice. By Lemma 2.7, two adjacent unread letters  $u < v$  above  $m$  lie in different intervals exactly when some letter between them was read at a time when the minimum read so far was already below  $u$ , so the stack is recovered from the scanned prefix alone. Adjacency cannot be dropped: after the prefix 3, 5, 1 of a 1342-avoiding permutation of [7] the stack is  $\{2, 4\} \mid \{6, 7\}$ , so 2 and 6 lie in different intervals, although no letter between them was read after a letter smaller than 2.

**Example 2.9** (A one-threshold scan). We scan the whole permutation  $\pi$  of (4), which can be read down the second column of the table below. The table lists, after each letter  $\pi_i$ , the type of move, the current minimum  $m$ , the interval stack, and the symbol  $W_p(L)$  of Section 2.3, where  $p$  is the number of unread letters below  $m$ ,  $L$  lists the interval sizes, and  $W_p(L)$  is the number of 1342-avoiding completions of the prefix. Every new minimum is a merger in the sense of Definition 2.4(a). The table writes “new minimum, merger” when the new minimum joins values to the head or creates the stack and “new minimum” alone when  $E = \emptyset$ . It writes “head exhausted” for an endpoint choice in a head of size one.

| $i$ | $\pi_i$ | move                | $m$ | interval stack                           | $W_p(L)$         |
|-----|---------|---------------------|-----|------------------------------------------|------------------|
| 1   | 9       | new minimum, merger | 9   | $\{10, \dots, 15\}$                      | $W_8((6))$       |
| 2   | 11      | interior choice     | 9   | $\{10\} \mid \{12, \dots, 15\}$          | $W_8((1, 4))$    |
| 3   | 10      | head exhausted      | 9   | $\{12, \dots, 15\}$                      | $W_8((4))$       |
| 4   | 14      | interior choice     | 9   | $\{12, 13\} \mid \{15\}$                 | $W_8((2, 1))$    |
| 5   | 5       | new minimum, merger | 5   | $\{6, 7, 8, 12, 13\} \mid \{15\}$        | $W_4((5, 1))$    |
| 6   | 12      | interior choice     | 5   | $\{6, 7, 8\} \mid \{13\} \mid \{15\}$    | $W_4((3, 1, 1))$ |
| 7   | 6       | endpoint choice     | 5   | $\{7, 8\} \mid \{13\} \mid \{15\}$       | $W_4((2, 1, 1))$ |
| 8   | 2       | new minimum, merger | 2   | $\{3, 4, 7, 8\} \mid \{13\} \mid \{15\}$ | $W_1((4, 1, 1))$ |
| 9   | 3       | endpoint choice     | 2   | $\{4, 7, 8\} \mid \{13\} \mid \{15\}$    | $W_1((3, 1, 1))$ |
| 10  | 8       | endpoint choice     | 2   | $\{4, 7\} \mid \{13\} \mid \{15\}$       | $W_1((2, 1, 1))$ |
| 11  | 4       | endpoint choice     | 2   | $\{7\} \mid \{13\} \mid \{15\}$          | $W_1((1, 1, 1))$ |
| 12  | 7       | head exhausted      | 2   | $\{13\} \mid \{15\}$                     | $W_1((1, 1))$    |
| 13  | 1       | new minimum         | 1   | $\{13\} \mid \{15\}$                     | $W_0((1, 1))$    |
| 14  | 13      | head exhausted      | 1   | $\{15\}$                                 | $W_0((1))$       |
| 15  | 15      | head exhausted      | 1   | $\emptyset$                              | $W_0(\emptyset)$ |

The first five rows are Examples 2.3 and 2.6. In the terms of Lemma 2.7, the letters 9, 10, and 11 separate nothing at row 5, as in Example 2.6: the separation produced by 11 at row 2 disappeared once the interval below it was exhausted, and the merger extended the head past it. The letter 14, read after  $9 < 13$ , separates 13 from 15 from row 4 on. The interior choice 12 then creates three intervals, separated between 8 and 13 and between 13 and 15. After the endpoint choice 6, the new minimum 2 joins  $\{3, 4\}$  to the head. The head  $\{3, 4, 7, 8\}$  contains the read letters 5 and 6 in its range, again without a separation. At row 8 the unread letters above  $m = 2$  are 3, 4, 7, 8, 13, 15, and the lemma cuts this list between 8 and 13, with witness 12 read after  $5 < 8$ , and between 13 and 15, with witness 14, and nowhere else. This is the stack of the table. The head is then consumed in the 231-avoiding order 3, 8, 4, 7. The new minimum 1 joins nothing

to the head, and 13, 15 complete the scan. Throughout, the later minima 5, 2, and 1 are read between letters of the stack, and only sizes enter the state: the heads  $\{6, 7, 8, 12, 13\}$  of row 5 and  $\{3, 4, 7, 8\}$  of row 8 contribute to  $L$  only their sizes 5 and 4.

**2.3. The one-threshold recurrence.** Recall that  $m$  is the least letter in the scanned prefix, virtual for the empty prefix. Let  $p$  be the number of unread letters below  $m$ , which we call the *control*, and let  $L = (\ell_1, \dots, \ell_s)$  be the positive sizes of the active and deferred intervals. We call  $(p, L)$  the *state* of the prefix. We write  $|L| = \sum_{i=1}^s \ell_i$  and  $\text{len}(L) = s$ . If  $L'$  is a list,  $(\ell)L'$  and  $(a, b)L'$  denote concatenation with the displayed initial entries, and  $\text{nz}(x_1, \dots, x_r)$  denotes the list obtained by deleting every zero entry. In the equations below, each summand stands for reading one letter and moving from the current state to the state indexed by the summand. We call such a move a *transition*.

**Proposition 2.10** (One-threshold recurrence). *For  $p \geq 0$  and every list  $L$  of positive integers, define  $W_p(L)$  as follows. For  $L = (\ell)L'$  with  $\ell \geq 1$ ,*

$$W_p((\ell)L') = \sum_{h=0}^{p-1} W_h((\ell + p - 1 - h)L') + \sum_{\substack{a, b \geq 0 \\ a+b=\ell-1}} W_p(\text{nz}(a, b)L'), \quad (5)$$

and

$$W_p(\emptyset) = \begin{cases} 1, & p = 0, \\ W_{p-1}(\emptyset) + \sum_{h=0}^{p-2} W_h((p-1-h)), & p \geq 1. \end{cases} \quad (6)$$

For every  $p$  and  $L$ ,  $W_p(L)$  is the number of maximal transition sequences from  $(p, L)$ , all of which end at  $(0, \emptyset)$ . For every legal prefix with state  $(p, L)$ , reading the letters of a completion one at a time is a bijection from the completions that produce a 1342-avoiding permutation to the maximal transition sequences from  $(p, L)$ . In particular

$$|\text{Av}_n(1342)| = W_n(\emptyset). \quad (7)$$

*Proof.* The quantity

$$\rho_1(p, L) = p + |L| \quad (8)$$

decreases by one under every transition, and  $(0, \emptyset)$  is the only state without transitions. Therefore the equations define  $W_p(L)$ , and by induction on  $\rho_1$  it is the number of maximal transition sequences from  $(p, L)$ , all of which end at  $(0, \emptyset)$ .

By Lemma 2.5(a), a legal prefix with state  $(p, L)$  has  $\rho_1(p, L)$  unread letters, and we argue by induction on this number. If no letter is unread, the prefix is a permutation, it avoids 1342 by Proposition 2.8(a), and its state is  $(0, \emptyset)$ , from which the only maximal sequence is empty. Otherwise we partition the possible next letters. By Proposition 2.8(b), an illegal letter begins no avoiding completion, and the legal letters are the new minima and the letters of the active head. If the next letter is below  $m$ , let  $h$  be the number of unread letters below it. There is one letter for each  $0 \leq h < p$ . The new minimum leaves  $h$  unrestricted lower letters, while the  $p-1-h$  letters between it and  $m$  join the active head. This gives the first sum in (5). A letter of the active head of local rank  $r$  replaces the head by its parts below and above it, of sizes  $a = r-1$  and  $b = \ell - r$ , so that  $L$  becomes  $\text{nz}(a, b)L'$  by Definition 2.4(b). This gives the last sum in (5), one letter for each pair  $(a, b)$  with  $a + b = \ell - 1$ . With an empty stack only a new minimum can be read, which gives (6). In every case the letter is recovered from its transition by its local rank, and the induction hypothesis applies to the successor. For the empty prefix,  $p = n$  and  $L = \emptyset$ , which is (7).  $\square$

**Example 2.11** (All one-threshold moves). After the prefix 9, 11, 10, 14, 5, 12 of the running example (4), row 6 of the table in Example 2.9, the unread letters below the current minimum

5 are 1, 2, 3, 4 and the stack is  $\{6, 7, 8\} \mid \{13\} \mid \{15\}$ . The state is  $(p, L) = (4, (3, 1, 1))$ , and (5) gives, with the two endpoint terms collected,

$$W_4((3, 1, 1)) = W_0((6, 1, 1)) + W_1((5, 1, 1)) + W_2((4, 1, 1)) + W_3((3, 1, 1)) \\ + 2W_4((2, 1, 1)) + W_4((1, 1, 1, 1)).$$

The first four terms correspond to reading 1, 2, 3, or 4, which join  $\{2, 3, 4\}$ ,  $\{3, 4\}$ ,  $\{4\}$ , or nothing to the active head. The fifth term records the two endpoints 6, 8 of the active head, and the last records its interior letter 7. Neither 13 nor 15 can be read legally until the active head is empty. The scan of [Example 2.9](#) continues with the endpoint 6.

**Example 2.12** (Counting  $\text{Av}_5(1342)$  by the literal recurrence). By (7) and (6),

$$|\text{Av}_5(1342)| = W_5(\emptyset) = W_0((4)) + W_1((3)) + W_2((2)) + W_3((1)) + W_4(\emptyset),$$

the five terms corresponding to the first letters 1, 2, 3, 4, 5. Expanding by (5) and (6) reaches twenty distinct states in all, listed here with their values, in increasing order of  $\rho_1 = p + |L|$ :

| $\rho_1$ | values                                                                                                            |
|----------|-------------------------------------------------------------------------------------------------------------------|
| 0        | $W_0(\emptyset) = 1$                                                                                              |
| 1        | $W_0((1)) = 1, W_1(\emptyset) = 1$                                                                                |
| 2        | $W_0((1, 1)) = 1, W_0((2)) = 2, W_1((1)) = 2, W_2(\emptyset) = 2$                                                 |
| 3        | $W_0((1, 2)) = 2, W_0((2, 1)) = 2, W_0((3)) = 5, W_1((1, 1)) = 3, W_1((2)) = 6, W_2((1)) = 6, W_3(\emptyset) = 6$ |
| 4        | $W_0((4)) = 14, W_1((3)) = 20, W_2((2)) = 23, W_3((1)) = 23, W_4(\emptyset) = 23$                                 |
| 5        | $W_5(\emptyset) = 103$                                                                                            |

For example,

$$W_0((4)) = 2W_0((3)) + W_0((1, 2)) + W_0((2, 1)) = 10 + 2 + 2 = 14,$$

$$W_1((3)) = W_0((3)) + 2W_1((2)) + W_1((1, 1)) = 5 + 12 + 3 = 20,$$

$$W_2((2)) = W_0((3)) + W_1((2)) + 2W_2((1)) = 5 + 6 + 12 = 23,$$

$$W_3((1)) = W_0((3)) + W_1((2)) + W_2((1)) + W_3(\emptyset) = 5 + 6 + 6 + 6 = 23,$$

so that  $W_5(\emptyset) = 14 + 20 + 23 + 23 + 23 = 103$ . In the first line the terms are the two endpoints and the interior choices  $(a, b) = (1, 2), (2, 1)$ , and in the second the new minimum, which joins nothing to the head, the two endpoints, and the interior choice  $(1, 1)$ . The values  $W_0((\ell))$  are the Catalan numbers  $C_\ell = \frac{1}{\ell+1} \binom{2\ell}{\ell}$ , as [Lemma 2.2](#) predicts. The number of distinct states grows exponentially with  $n$  ([Proposition 2.13](#) below). [Example 2.18](#) recomputes the same value from the kernel rows of the shrunk state space.

**2.4. The exponential literal state space.** The recurrence (5) is finite for each  $n$ , but memoizing its composition-valued argument does not give a polynomial bound.

**Proposition 2.13.** *For  $n \geq 2$ , the computation of  $W_n(\emptyset)$  reaches at least  $F_{n-1}$  distinct composition arguments, where  $F_{n-1}$  is the  $(n-1)$ st Fibonacci number,  $F_1 = F_2 = 1$ .*

*Proof.* Choosing the letter 1 at the initial state reaches  $W_0((n-1))$ . Repeated interior choices reach every positive composition  $L$  satisfying

$$|L| + \text{len}(L) = n. \tag{9}$$

Reversing an interior split replaces  $(a, b, \dots)$  by  $(a + b + 1, \dots)$ , and repeated reversals return every such composition to  $(n-1)$ . If  $L$  has  $s$  parts, then  $|L| = n - s$ , so the number of choices is  $\binom{n-s-1}{s-1}$ . Summing over  $1 \leq s \leq \lfloor n/2 \rfloor$  gives

$$\sum_{s=1}^{\lfloor n/2 \rfloor} \binom{n-s-1}{s-1} = F_{n-1}. \quad \square$$

Literal memoization of (5) therefore has exponentially many states. The values of the ordered parts cannot be replaced by their total and their number. For example, when  $p = 0$ , (3) gives  $W_0((1, 3)) = C_1 C_3 = 5$  and  $W_0((2, 2)) = C_2^2 = 4$ . The next construction removes the entire composition from the memoization key while retaining its effect on completion counts.

**2.5. Protected-tail factorization.** Before the general statement, consider the smallest nontrivial instance of (5): one unread letter below the current minimum and a head of size one, with an arbitrary tail  $L'$ . Either the head letter is read first, which leaves the stack  $L'$  at control 1, or the lower letter is read first and the head letter next, which leaves  $L'$  at control 0. Therefore

$$W_1((1)L') = W_1(L') + W_0(L').$$

The two coefficients count the ways to reach the stack  $L'$  at controls 1 and 0, and they do not depend on  $L'$ . Writing them as  $K_1(1, 1) = K_1(1, 0) = 1$ , the identity reads  $W_1((1)L') = \sum_{t \in \{0, 1\}} K_1(1, t) W_t(L')$ . The theorem below shows that such a factorization holds for every head size: the transitions before the tail is exposed leave the tail unchanged and do not depend on it.

For a stack  $UL$  we write  $U \mid L$  when the suffix  $L$  is the one under consideration. The marker is a notational device and not part of the state. A state is an *exposure* of  $L$  if its stack is exactly  $L$ , that is, if it has the form  $(t, \emptyset \mid L)$ . A *path* is a finite sequence of consecutive transitions. For a nonempty list  $U$ , a path from  $(p, U \mid L)$  is *stopped* if it ends at an exposure of  $L$  and contains no earlier one, and  $K_U^L(p, t)$  denotes the number of stopped paths from  $(p, U \mid L)$  to  $(t, \emptyset \mid L)$ . We write  $K_\ell^L = K_{(\ell)}^L$ , and we let  $K_\emptyset$  be the identity kernel  $\mathbf{1}_{p=t}$ , which is 1 if  $p = t$  and 0 otherwise. Kernels are multiplied as matrices,  $(AB)(p, t) = \sum_{u \geq 0} A(p, u) B(u, t)$ .

**Theorem 2.14** (Protected-tail factorization). *For every nonempty list  $U$  of positive integers, the number  $K_U^L(p, t)$  is independent of  $L$ . We write  $K_U(p, t)$  for it. Then, for all lists  $U, V$  and  $L$  of positive integers,*

$$W_p(UL) = \sum_{t \geq 0} K_U(p, t) W_t(L) \quad \text{and} \quad K_{UV} = K_U K_V. \quad (10)$$

In particular  $K_{(\ell_1, \dots, \ell_s)} = K_{\ell_1} \cdots K_{\ell_s}$ , and for  $L = (\ell_1, \dots, \ell_s)$

$$W_p(L) = \sum_{t \geq 0} (K_{\ell_1} K_{\ell_2} \cdots K_{\ell_s})(p, t) W_t(\emptyset). \quad (11)$$

*Proof.* Both identities are trivial when  $U$  is empty, and the second also when  $V$  is empty, so let  $U$  and  $V$  be nonempty. Before  $L$  is exposed, a state has the form  $(p', U' \mid L)$  with  $U' \neq \emptyset$ . By (5), its transitions change only the control and the blocks before the marker, and they and their targets are the same for every choice of  $L$ . Replacing  $L$  by another list therefore maps the stopped paths from  $(p, U \mid L)$  bijectively onto those for the other list, with the same controls, so  $K_U^L(p, t)$  is independent of  $L$ . By Proposition 2.10,  $W_p(UL)$  is the number of maximal transition sequences from  $(p, UL)$ . Each of them ends at  $(0, \emptyset)$ , whose stack is empty, and changes only the blocks before the marker until  $L$  is exposed, so it passes through a first exposure  $(t, \emptyset \mid L)$  of  $L$ . Cutting it there gives a stopped path counted by  $K_U(p, t)$  followed by a maximal sequence from  $(t, L)$ , counted by  $W_t(L)$ , and concatenation is the inverse operation. In the same way, a stopped path from  $(p, UV \mid L)$  leaves  $VL$  unchanged until  $U$  is used up, so it passes through a first exposure  $(u, \emptyset \mid VL)$  of  $VL$ , and cutting it there gives a stopped path from  $(p, U \mid VL)$  followed by one from  $(u, V \mid L)$ , so  $K_{UV} = K_U K_V$ . Taking  $U = (\ell_1, \dots, \ell_s)$  and  $L = \emptyset$  in the first identity and applying the second repeatedly gives (11).  $\square$

*Remark 2.15* (Pushdown systems and pop sequences). The proof uses two properties of (5): before the marked tail is exposed, the transitions depend only on the control and the blocks before the marker and leave the tail unchanged, and every maximal transition sequence exposes the tail. They

say that the recurrence is a pushdown system. The kernel  $K_\ell(p, t)$  counts the runs that pop the block  $\ell$  while passing from  $p$  to  $t$ , and [Theorem 2.14](#) is the counting form of the classical conversion of a pushdown automaton into a context-free grammar, whose variable  $[p \ell t]$  describes exactly those runs, see Hopcroft, Motwani, and Ullman [18, Theorem 6.14]; Sipser [23, Lemma 2.27] gives a variant with variables  $A_{pq}$ . The same runs are the pop sequences of the weighted pushdown systems of Reps, Schwoon, Jha, and Melski [22], and (11) is the first-passage decomposition of weighted lattice paths, see Flajolet and Sedgewick [15, Sections I.5.3 and V.4]. With infinitely many controls the construction gives an infinite grammar, so the Chomsky–Schützenberger theorem [12] does not apply and no algebraicity follows, and what the factorization provides is a memoization key of polynomial size ([Theorem 1.1](#)) rather than a generating function. The same two properties hold for the recurrence of general  $d$ , and [Section 5](#) applies the proof there.

**2.6. The scalar kernel algorithm.** Recall that the control  $p$  is the number of unread letters below the current minimum. For a kernel entry  $K_\ell(p, t)$  of [Theorem 2.14](#) we call  $p$  the *source control* and  $t$  the *terminal control*, and we call  $K_\ell(p, \cdot)$  a *scalar kernel row*. A transition that selects an unread letter below the current minimum is a *base move*. A transition that selects a value from the active head is an *active-head move*. Put  $K_0 = K_\emptyset$ , the identity kernel, and  $G_p = W_p(\emptyset)$ .

**Proposition 2.16** (Scalar kernel recurrences). *For  $\ell \geq 1$ ,*

$$K_\ell(p, t) = \sum_{h=0}^{p-1} K_{\ell+p-1-h}(h, t) + \sum_{\substack{a, b \geq 0 \\ a+b=\ell-1}} \sum_{u \geq 0} K_a(p, u) K_b(u, t). \quad (12)$$

*The empty-stack values satisfy*

$$G_p = \begin{cases} 1, & p = 0, \\ \sum_{h=0}^{p-1} \sum_{t \geq 0} K_{p-1-h}(h, t) G_t, & p \geq 1, \end{cases} \quad |\text{Av}_n(1342)| = G_n. \quad (13)$$

*Proof.* We partition the stopped paths from  $(p, (\ell) \mid L)$  by their first move. A new minimum leaving  $h$  lower letters gives the first sum in (12). A letter of the active head of local rank  $r$  leaves the stack  $\text{nz}(a, b)L$  with  $a = r - 1$  and  $b = \ell - r$ , so the rest of the path is a stopped path from  $(p, \text{nz}(a, b) \mid L)$ , or the empty path when  $\ell = 1$ . These are counted by  $K_{\text{nz}(a, b)}(p, t) = (K_a K_b)(p, t)$ , by [Theorem 2.14](#), which gives the double sum. Finally, the recurrence in (13) follows from (6): a new minimum leaving  $h$  lower letters creates a part of size  $p - 1 - h$ , to which (10) applies, and  $K_0$  covers the case  $h = p - 1$ . The last equation is (7).  $\square$

**Lemma 2.17** (Exact scalar support). *For every  $p \geq 0$  and  $\ell \geq 1$ , the support of the row  $K_\ell(p, \cdot)$ , the set of  $t$  with  $K_\ell(p, t) \neq 0$ , is*

$$\text{supp } K_\ell(p, \cdot) = \{0, 1, \dots, p\}. \quad (14)$$

*In addition,  $K_\ell(p, p) = C_\ell$ .*

*Proof.* A base move strictly decreases the control, while an active-head move leaves it unchanged. Thus every terminal control is at most  $p$ . A path ending at  $p$  uses no base move. By [Lemma 2.2](#), such paths are in bijection with the 231-avoiding permutations of the original  $\ell$ -element interval, so there are  $C_\ell$  of them.

For  $t < p$ , consider the path that first reads the unread letter below the current minimum that has  $t$  unread letters below it, which makes the control  $t$  and the active head of size  $\ell + p - 1 - t$ , and then reads endpoints until the head is exhausted. It makes no other base move and exposes the marked suffix at control  $t$ . Thus every  $t \in \{0, \dots, p\}$  occurs.  $\square$

We call  $p + \ell$  the *grade* of the kernel row  $K_\ell(p, \cdot)$ .

**Example 2.18** (From kernel rows to  $|\text{Av}_5(1342)|$ ). We write each row  $K_\ell(p, \cdot)$  as the vector  $(K_\ell(p, 0), \dots, K_\ell(p, p))$ , which by Lemma 2.17 lists its whole support. Equation (12) gives

$$K_1(1, \cdot) = (1, 1), \quad K_3(0, \cdot) = (5), \quad K_2(1, \cdot) = (4, 2), \quad K_1(2, \cdot) = (3, 1, 1),$$

for example  $K_1(2, \cdot) = K_2(0, \cdot) + K_1(1, \cdot) + \mathbf{e}_2 = (2) + (1, 1) + \mathbf{e}_2 = (3, 1, 1)$ , where  $\mathbf{e}_t(u) = \mathbf{1}_{u=t}$ . The rows of grade 4 follow:

$$\begin{aligned} K_4(0, \cdot) &= 2K_3(0, \cdot) + K_1(0, 0)K_2(0, \cdot) + K_2(0, 0)K_1(0, \cdot) = (10) + (2) + (2) = (14), \\ K_3(1, \cdot) &= K_3(0, \cdot) + 2K_2(1, \cdot) + \sum_u K_1(1, u)K_1(u, \cdot) = (5) + (8, 4) + (2, 1) = (15, 5), \\ K_2(2, \cdot) &= K_3(0, \cdot) + K_2(1, \cdot) + 2K_1(2, \cdot) = (5) + (4, 2) + (6, 2, 2) = (15, 4, 2), \\ K_1(3, \cdot) &= K_3(0, \cdot) + K_2(1, \cdot) + K_1(2, \cdot) + \mathbf{e}_3 = (12, 3, 1, 1). \end{aligned}$$

Since  $(G_0, \dots, G_4) = (1, 1, 2, 6, 23)$ , (13) gives

$$\begin{aligned} G_5 &= 14G_0 + (15G_0 + 5G_1) + (15G_0 + 4G_1 + 2G_2) + (12G_0 + 3G_1 + G_2 + G_3) + G_4 \\ &= 14 + 20 + 23 + 23 + 23 = 103, \end{aligned}$$

the five groups corresponding, in order, to  $h = 0, 1, 2, 3, 4$ , the last through the identity kernel  $K_0$ . The five group sums are the five terms  $W_0((4)), \dots, W_4(\emptyset)$  of Example 2.12: each kernel row compresses the subtree of the literal recurrence below one first-letter choice into a vector indexed by the control at which the empty tail is exposed.

**Theorem 2.19** (Polynomial enumeration of 1342-avoiders). *For every  $N \geq 1$ , the numbers  $|\text{Av}_n(1342)|$  for  $0 \leq n \leq N$  can be computed using  $O(N^5)$  integer arithmetic operations and  $O(N^3)$  stored integers. Every integer has  $O(N \log N)$  bits. Therefore the bit complexity is  $O(N^5 M(N \log N))$  and the bit storage is  $O(N^4 \log N)$ , where  $M(b)$  is the cost of multiplying  $b$ -bit integers, as in Theorem 1.1.*

*Proof.* We order the kernel rows by grade. The row  $K_{\ell+p-1-h}(h, \cdot)$  in a base term of (12) has grade  $p + \ell - 1$ . In a split term,  $p + a < p + \ell$ . If  $K_a(p, u) \neq 0$ , then  $u \leq p$ , by Lemma 2.17 when  $a \geq 1$  and since  $K_0$  is the identity when  $a = 0$ , so that  $u + b \leq p + b < p + \ell$ . Thus increasing grade is a topological order for all kernel dependencies: an order in which every row comes after all the rows its recurrence uses, so that evaluating the rows in this order never meets a row not yet computed. Equation (13) is then evaluated in increasing  $p$ , because every terminal control  $t$  in its nonzero terms satisfies  $t \leq h < p$ .

We compute every row in the *padded region*  $p \geq 0, \ell \geq 1, p + \ell \leq N$ : it is closed under the dependencies in the preceding paragraph and contains every row with  $\ell \geq 1$  needed to evaluate (13) for  $n \leq N$ , together with rows that no such evaluation reaches. It contains  $\binom{N+1}{2} = O(N^2)$  source rows. By Lemma 2.17, these rows have at most

$$\sum_{\ell=1}^N \sum_{p=0}^{N-\ell} (p+1) = \binom{N+2}{3} = O(N^3)$$

nonzero entries in total. For one source row, a sparse evaluation of all split terms has  $O(N)$  choices of  $a$ ,  $O(N)$  possible intermediate controls  $u$ , and  $O(N)$  terminal controls in the row being added. It therefore uses  $O(N^3)$  operations per source row and  $O(N^5)$  operations in total. The nonsplit kernel terms use  $O(N^4)$  operations in total, and (13) uses  $O(N^3)$ .

For a scalar kernel entry, we take the marked suffix to be empty, by Theorem 2.14. Its initial number of unread letters is then  $p + \ell \leq N$ . An empty-stack computation also starts with at most  $N$  unread letters. This number decreases at every transition. At each state, the number of outgoing transitions is the number of legal next letters and is at most  $N$ . Thus each stored count is at most  $(N+1)^N$  and has  $O(N \log N)$  bits. Accounting for integer multiplication and

for the bit length of every stored entry gives the stated bit bounds. Every product and partial accumulator is a nonnegative contribution to a final kernel or empty-stack value, so the same bound applies to intermediate integers.  $\square$

Bóna [10] found the algebraic generating function

$$\sum_{n \geq 0} |\text{Av}_n(1342)| x^n = \frac{(1 - 8x)^{3/2} - 8x^2 + 20x + 1}{2(1 + x)^3}. \quad (15)$$

[Theorem 2.19](#) is a separate transfer construction, and it extends to the family: we replace  $p$  by  $d$  ordered band counts, while the interval stack, its merger rule, and the protected-tail theorem remain unchanged.

### 3. FROM ONE THRESHOLD TO MANY

We now consider the pattern  $\beta_d$  for an arbitrary  $d \geq 1$ , and replace the current minimum of [Section 2](#) by  $d$  thresholds and the bands of unread values below and between them. After a prefix has been read, let  $b_j$  be the smallest final entry of an increasing subsequence of length  $j$  contained in that prefix, for  $1 \leq j \leq d$ . When the prefix contains no such subsequence, we use the virtual value  $b_j = n + j$ . Then

$$b_1 < b_2 < \cdots < b_d,$$

since deleting the last entry of an increasing  $(j + 1)$ -subsequence ending at a nonvirtual  $b_{j+1}$  leaves one of length  $j$  ending below  $b_{j+1}$ , and a virtual  $b_{j+1} = n + j + 1$  exceeds  $b_j \leq n + j$ . We call  $b_1, \dots, b_d$  the *patience-sorting thresholds* of the prefix, since they are the top cards of the first  $d$  piles when the prefix is dealt by patience sorting, see Aldous and Diaconis [4]. Every nonvirtual threshold value has already been read. We call the unread values below  $b_d$  the *base values*. They fall into  $d$  open bands, shown in [Figure 3](#). Band 0 lies below  $b_1$ , while band  $i$  lies between  $b_i$  and  $b_{i+1}$  for  $1 \leq i < d$ . We write  $\mathbf{p} = (p_0, p_1, \dots, p_{d-1})$  for the band sizes. We again call  $\mathbf{p}$  the *control*, write  $\|\mathbf{p}\|_1 = p_0 + \cdots + p_{d-1}$  for its *control mass*, and  $\mathbf{0}$  for the zero control. Let  $B_0, \dots, B_{d-1}$  be the corresponding unread blocks. Together with the intervals of the stack, the value order is

$$B_0 < b_1 < B_1 < b_2 < \cdots < b_{d-1} < B_{d-1} < b_d < I_1 < \cdots < I_s \quad (16)$$

by [Proposition 4.1\(a\)](#) below. Empty blocks are allowed. As in the one-threshold case ([Section 2](#)), a base move selects a value from some  $B_i$  and an active-head move selects a value from  $I_1$ . We call a base move from  $B_i$  with  $i < d - 1$  an *early-band move* and a base move from  $B_{d-1}$  a *last-band move*. Initially  $\mathbf{p} = (n, 0, \dots, 0)$ .

If the next base value  $x$  lies in band  $i \geq 1$ , then  $x$  ends an increasing subsequence of length  $i + 1$ , since  $b_i < x$ . For  $i = 0$ , the letter  $x$  is an increasing subsequence of length one. In either case,  $x$  ends no increasing subsequence of length  $i + 2$ , since that would imply  $b_{i+1} < x$ , while  $x \in B_i$  gives  $x < b_{i+1}$ . Thus only  $b_{i+1}$  changes. A base value creates a new  $d$ -trigger exactly when it lies in band  $d - 1$ . The first  $d - 1$  bands update one threshold without creating an obligation. The last band also creates a 231-avoidance obligation.

Since a  $d$ -trigger is the last entry of an increasing  $d$ -subsequence,  $b_d$  is the least  $d$ -trigger of the prefix when it is nonvirtual, and a virtual  $b_d = n + d$  means that the prefix has no  $d$ -trigger. With  $q$  as in [Definition 2.4](#), in both cases

$$b_d = q. \quad (17)$$

In particular, the unread values below  $b_d$  belong to the projection of no trigger read so far, while every unread value above a nonvirtual  $b_d$  belongs to the projection created by the trigger  $b_d$ . In the terms of [Definition 2.4](#), a value of  $B_i$  with  $i < d - 1$  is a non-trigger move, a value of  $B_{d-1}$  is a merger, and a value of  $I_1$  is a split ([Proposition 4.1\(b\)](#) below).

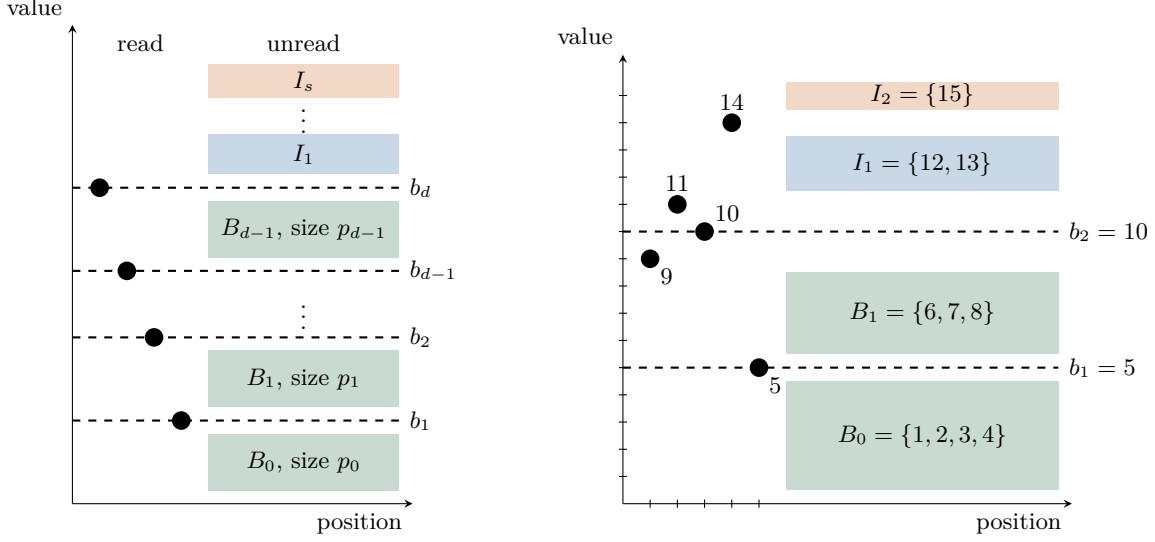


FIGURE 3. The bands and the stack. Left, schematically: the thresholds  $b_1 < \dots < b_d$  (the only read letters drawn, at arbitrary positions), the bands  $B_0, \dots, B_{d-1}$  of unread base values with sizes  $\mathbf{p} = (p_0, \dots, p_{d-1})$ , and the stack  $I_1, \dots, I_s$  above  $b_d$ . Right: the running example (4) for  $d = 2$  after the prefix 9, 11, 10, 14, 5, with  $b_1 = 5$ ,  $b_2 = 10$ ,  $\mathbf{p} = (4, 3)$ , and the stack  $\{12, 13\} \mid \{15\}$ , as in row 5 of the table in Example 4.2.

For example, in the permutation 316829574 the entries 3, 6 form an increasing 2-subsequence, and the subsequent projection onto values larger than 6 is 8, 9, 7, an occurrence of 231. Therefore 3, 6, 8, 9, 7 is an occurrence of  $\beta_2 = 12453$ . The interspersed values 2, 5, 4, all smaller than 6, do not belong to this projection.

#### 4. THE LITERAL FAMILY RECURRENCE

In this section we write down the literal recurrence for  $\beta_d$ , show that it counts every avoider once, characterize the separations of its stack, and show that its state space is exponential. We now express the band updates in terms of their sizes. For  $0 \leq i < d - 1$  and  $0 \leq h < p_i$ , let  $x$  be the value of  $B_i$  that has  $h$  unread values of  $B_i$  below it. Reading  $x$  makes it the new threshold  $b_{i+1}$ , leaves  $h$  values in band  $i$ , and moves the  $p_i - 1 - h$  values of  $B_i$  above it to band  $i + 1$ , so the control becomes

$$T_{i,h}(\mathbf{p}) = (p_0, \dots, p_{i-1}, h, p_{i+1} + p_i - 1 - h, p_{i+2}, \dots, p_{d-1}). \quad (18)$$

Similarly, if  $x$  is the value of  $B_{d-1}$  that has  $h$  unread values of  $B_{d-1}$  below it, reading  $x$  gives the control

$$U_h(\mathbf{p}) = (p_0, \dots, p_{d-2}, h), \quad \delta_h = p_{d-1} - 1 - h. \quad (19)$$

For  $d = 1$ , interpret (19) as  $U_h((p_0)) = (h)$ . Here  $x$  is a new least  $d$ -trigger, and the  $\delta_h$  unread values above it and below the old  $b_d$  join the active head.

For example, take  $d = 2$  and the prefix 9, 11, 10, 14, 5 of the running example (4), shown on the right of Figure 3, with  $b_1 = 5$ ,  $b_2 = 10$ , and  $\mathbf{p} = (4, 3)$ . Reading 2, which has  $h = 1$  unread value of  $B_0 = \{1, 2, 3, 4\}$  below it, makes 2 the new  $b_1$ , leaves  $\{1\}$  in band 0, and moves 3, 4 into band 1, so the control becomes  $T_{0,1}(\mathbf{p}) = (1, 5)$ . Reading 7 instead, which has  $h = 1$  unread value of  $B_1 = \{6, 7, 8\}$  below it, makes 7 the new  $b_2$  and gives the control  $U_1(\mathbf{p}) = (4, 1)$ , and the  $\delta_1 = 1$

value 8 joins the head, so the stack becomes  $\{8, 12, 13\} \mid \{15\}$ . [Example 4.2](#) follows the whole running example through such moves.

Let  $L = (\ell_1, \dots, \ell_s)$  be the sizes of the intervals of the stack, active head first, and call  $(\mathbf{p}, L)$  the *state* of a legal prefix ([Definition 2.4](#)). For an empty stack we interpret  $\ell_1 = 0$  and its tail as empty.

**Proposition 4.1** (State invariant). *Let  $\sigma$  be a legal prefix with thresholds  $b_1 < \dots < b_d$ , band sizes  $\mathbf{p}$ , and stack  $\mathcal{S}(\sigma) = (I_1, \dots, I_s)$  with sizes  $L = (\ell_1, \dots, \ell_s)$ .*

- (a) *The stack consists of the unread values above  $b_d$ , so the ordered layout (16) holds.*
- (b) *The legal moves from  $\sigma$ , and their effect on  $(\mathbf{p}, L)$ , are the following. A value of  $B_i$  with  $i < d-1$  having  $h$  unread values of  $B_i$  below it is a non-trigger move to  $(T_{i,h}(\mathbf{p}), L)$ . A value of  $B_{d-1}$  having  $h$  unread values of  $B_{d-1}$  below it is a merger to  $(U_h(\mathbf{p}), \text{nz}(\ell_1 + \delta_h, \ell_2, \dots))$ . A value of  $I_1$  of local rank  $r$  is a split to  $(\mathbf{p}, \text{nz}(r-1, \ell_1 - r, \ell_2, \dots))$ . No other letter is legal.*

*Proof.* (a) Since  $b_d = q$  by (17), the stack consists of the unread values above  $b_d$  by [Lemma 2.5\(a\)](#), while the unread values below  $b_d$  form the bands by definition. This is (16).

(b) We classify the unread letters by [Definition 2.4](#). As observed before (17), a letter  $x \in B_i$  changes only  $b_{i+1}$ , and it is a non-trigger move if  $i < d-1$  and a merger if  $i = d-1$ , since then  $x < b_d = q$ . A letter of  $I_1$  exceeds  $b_d$ , so it changes no threshold, and it is a split, as noted after [Definition 2.4](#). The new controls are given by (18) and (19), the set  $E$  of a merger has  $\delta_h$  elements, and the new stacks follow from [Definition 2.4](#). Every other unread letter lies in a deferred interval and is illegal by [Lemma 2.5\(b\)](#).  $\square$

For a control  $\mathbf{p}$  and a list  $L$  of positive integers with  $(\mathbf{p}, L) \neq (\mathbf{0}, \emptyset)$ , we define

$$\begin{aligned} H_{\mathbf{p}}(L) &= \sum_{i=0}^{d-2} \sum_{h=0}^{p_i-1} H_{T_{i,h}(\mathbf{p})}(L) \\ &\quad + \sum_{h=0}^{p_{d-1}-1} H_{U_h(\mathbf{p})}(\text{nz}(\ell_1 + \delta_h, \ell_2, \dots)) \\ &\quad + \sum_{\substack{a,b \geq 0 \\ a+b=\ell_1-1}} H_{\mathbf{p}}(\text{nz}(a, b, \ell_2, \dots)). \end{aligned} \tag{20}$$

The three lines represent, respectively, an early-band move, a last-band move, and a letter of the active head, of local rank  $a+1$ , and as in [Section 2.3](#) each summand stands for a transition. The first double sum is empty when  $d=1$ , and the last line is empty when the stack is empty. The quantity

$$\rho(\mathbf{p}, L) = \|\mathbf{p}\|_1 + \sum_{j=1}^s \ell_j \tag{21}$$

drops by one on every transition. An early-band move reduces the control mass by one. A last-band move reduces it by  $\delta_h + 1$  and adds  $\delta_h$  to the head. An active-head move consumes one head value. The zero state  $(\mathbf{0}, \emptyset)$  is the only state without transitions: a nonzero control allows a base move and a nonempty stack a letter of the head. Therefore (20) and  $H_{\mathbf{0}}(\emptyset) = 1$  define  $H_{\mathbf{p}}(L)$ , and by induction on  $\rho$  it is the number of maximal transition sequences from  $(\mathbf{p}, L)$ , all of which end at the zero state. [Theorem 4.3](#) below shows that

$$a_n^{(d)} := |\text{Av}_n(\beta_d)| = H_{(n,0,\dots,0)}(\emptyset). \tag{22}$$

For  $d=1$ , identify  $H_{(p)}(L) = W_p(L)$ . The early-band sum is empty and  $U_h((p)) = (h)$ , so (20) and  $H_{\mathbf{0}}(\emptyset) = 1$  reduce to (5) and (6), (21) to (8), and (22) to (7).

**Example 4.2** (What the second threshold adds). We specialize to  $d = 2$  and scan the running example (4) again, which can be read down the second column of the table below. It avoids 12453, since 12453 contains 1342. The table lists, after each letter  $\pi_i$ , the type of move, the thresholds  $b_1, b_2$ , the bands  $B_0, B_1$ , the interval stack, and  $H_{\mathbf{p}}(L)$ , the number of 12453-avoiding completions. A dot marks a virtual threshold. Band 0 is the early band and band 1 the last band, and in the sense of Definition 2.4 an early-band move is a non-trigger move and a last-band move is a merger. As in Example 2.9, the table writes “merger” when the move joins values to the head or creates the stack, so that “last band” alone marks the case  $E = \emptyset$ , and it writes “head exhausted” for an endpoint choice in a head of size one.

| $i$ | $\pi_i$ | move              | $b_1, b_2$ | $B_0$             | $B_1$               | interval stack                     | $H_{\mathbf{p}}(L)$    |
|-----|---------|-------------------|------------|-------------------|---------------------|------------------------------------|------------------------|
| 1   | 9       | early band        | 9, ·       | $\{1, \dots, 8\}$ | $\{10, \dots, 15\}$ | $\emptyset$                        | $H_{(8,6)}(\emptyset)$ |
| 2   | 11      | last band, merger | 9, 11      | $\{1, \dots, 8\}$ | $\{10\}$            | $\{12, \dots, 15\}$                | $H_{(8,1)}((4))$       |
| 3   | 10      | last band         | 9, 10      | $\{1, \dots, 8\}$ | $\emptyset$         | $\{12, \dots, 15\}$                | $H_{(8,0)}((4))$       |
| 4   | 14      | interior choice   | 9, 10      | $\{1, \dots, 8\}$ | $\emptyset$         | $\{12, 13\} \mid \{15\}$           | $H_{(8,0)}((2, 1))$    |
| 5   | 5       | early band        | 5, 10      | $\{1, \dots, 4\}$ | $\{6, 7, 8\}$       | $\{12, 13\} \mid \{15\}$           | $H_{(4,3)}((2, 1))$    |
| 6   | 12      | endpoint choice   | 5, 10      | $\{1, \dots, 4\}$ | $\{6, 7, 8\}$       | $\{13\} \mid \{15\}$               | $H_{(4,3)}((1, 1))$    |
| 7   | 6       | last band, merger | 5, 6       | $\{1, \dots, 4\}$ | $\emptyset$         | $\{7, 8, 13\} \mid \{15\}$         | $H_{(4,0)}((3, 1))$    |
| 8   | 2       | early band        | 2, 6       | $\{1\}$           | $\{3, 4\}$          | $\{7, 8, 13\} \mid \{15\}$         | $H_{(1,2)}((3, 1))$    |
| 9   | 3       | last band, merger | 2, 3       | $\{1\}$           | $\emptyset$         | $\{4, 7, 8, 13\} \mid \{15\}$      | $H_{(1,0)}((4, 1))$    |
| 10  | 8       | interior choice   | 2, 3       | $\{1\}$           | $\emptyset$         | $\{4, 7\} \mid \{13\} \mid \{15\}$ | $H_{(1,0)}((2, 1, 1))$ |
| 11  | 4       | endpoint choice   | 2, 3       | $\{1\}$           | $\emptyset$         | $\{7\} \mid \{13\} \mid \{15\}$    | $H_{(1,0)}((1, 1, 1))$ |
| 12  | 7       | head exhausted    | 2, 3       | $\{1\}$           | $\emptyset$         | $\{13\} \mid \{15\}$               | $H_{(1,0)}((1, 1))$    |
| 13  | 1       | early band        | 1, 3       | $\emptyset$       | $\emptyset$         | $\{13\} \mid \{15\}$               | $H_{(0,0)}((1, 1))$    |
| 14  | 13      | head exhausted    | 1, 3       | $\emptyset$       | $\emptyset$         | $\{15\}$                           | $H_{(0,0)}((1))$       |
| 15  | 15      | head exhausted    | 1, 3       | $\emptyset$       | $\emptyset$         | $\emptyset$                        | $H_{(0,0)}(\emptyset)$ |

Compare with the one-threshold scan of Example 2.9. The first letter 9 is not a 2-trigger and creates no obligation. It becomes  $b_1$ , and the unread values above it form band 1. The first 2-trigger is 11, whose obligation constrains  $\{12, \dots, 15\}$ . The value 10 remains a base value, and reading it next makes it the least 2-trigger without joining anything to the head. The new minimum 5, a merger for  $d = 1$ , is here an early-band move: it transfers  $\{6, 7, 8\}$  from band 0 to band 1 and leaves the head  $\{12, 13\}$  unchanged. It is the 2-trigger 6, not the base value 5, that joins  $\{7, 8\}$  to the head. The read letters 9 to 12 lie inside the range of the head  $\{7, 8, 13\}$ , and by Corollary 4.4 below none of them separates 8 from 13: neither 9 nor 11 was read after an earlier 2-trigger smaller than 8, and 10 and 12 were read only after 2-triggers exceeding 8. The letter 14 separates 13 from 15, because it was read after the 2-trigger  $10 < 13$ . The letter 8, an endpoint choice for  $d = 1$ , is here an interior choice.

After the eighth letter, the thresholds are  $b_1 = 2$  and  $b_2 = 6$ , the bands are  $B_0 = \{1\}$  and  $B_1 = \{3, 4\}$ , and the stack is  $\{7, 8, 13\} \mid \{15\}$ . The state is  $(\mathbf{p}, L) = ((1, 2), (3, 1))$ . Its next choices are:

| next    | action                                          | successor                   |
|---------|-------------------------------------------------|-----------------------------|
| 1       | update the first threshold                      | $H_{(0,2)}((3, 1))$         |
| 3       | new 2-trigger, $\{4\}$ joins the head           | $H_{(1,0)}((4, 1))$         |
| 4       | new 2-trigger, no value is added to the head    | $H_{(1,1)}((3, 1))$         |
| 7 or 13 | endpoint choice in the active head              | $H_{(1,2)}((2, 1))$ , twice |
| 8       | interior choice with two singleton subintervals | $H_{(1,2)}((1, 1, 1))$      |
| 15      | illegal until the head is exhausted             | illegal                     |

Therefore

$$\begin{aligned} H_{(1,2)}((3,1)) &= H_{(0,2)}((3,1)) + H_{(1,0)}((4,1)) + H_{(1,1)}((3,1)) \\ &\quad + 2H_{(1,2)}((2,1)) + H_{(1,2)}((1,1,1)). \end{aligned}$$

The extra threshold is visible in the first row: the choice 1 updates  $b_1$  without creating a 2-trigger and, in this state, transfers no unread value to the second band. The values of an active head need not be read consecutively. Base values may occur between them. The scan continues with 3.

**Theorem 4.3** (Literal recurrence). *Let  $d \geq 1$ . For every legal prefix with state  $(\mathbf{p}, L)$ , reading the letters of a completion one at a time is a bijection from the completions that produce a  $\beta_d$ -avoiding permutation to the maximal sequences of transitions from  $(\mathbf{p}, L)$ , and  $H_{\mathbf{p}}(L)$  is their number. In particular (22) holds.*

*Proof.* By Proposition 4.1(a), the unread values of a legal prefix with state  $(\mathbf{p}, L)$  are the  $\rho(\mathbf{p}, L)$  values of its bands and its stack, and we argue by induction on this number, as in the proof of Proposition 2.10. If no value is unread, the prefix is a permutation, it avoids  $\beta_d$  by Proposition 2.8(a), and its state is the zero state. Otherwise, by Proposition 2.8(b) an illegal letter begins no avoiding completion, and by Proposition 4.1(b) the legal letters and their successor states are the terms of (20), one letter for each term. The induction hypothesis applies to the successors. The empty prefix has the state  $((n, 0, \dots, 0), \emptyset)$ .  $\square$

For  $d = 2$ , (20) is the recurrence evaluated by Biers-Ariel's program [8]. Its function `Avoid12453_helper(i,j,L)` is  $H_{(i-1,j-i-1)}(L)$ , with the list  $[0]$  for the empty stack, and its four contributions to the output are, in order, the early-band, last-band, endpoint and interior terms of (20).

The stack can be read off the scanned prefix. Recall that two unread values are adjacent if no unread value lies strictly between them. By Proposition 2.8(c), the legal prefixes are exactly the prefixes of  $\beta_d$ -avoiding permutations, and every legal prefix has an avoiding completion, so condition (iii) below is not vacuous.

**Corollary 4.4** (Separations and completions). *After a legal prefix has been read, let  $u < v$  be adjacent unread values above  $b_d$ . The following are equivalent.*

- (i)  $u$  and  $v$  lie in different intervals of the stack.
- (ii) Some letter  $x$  with  $u < x < v$  was read after a  $d$ -trigger smaller than  $u$ .
- (iii)  $u$  precedes  $v$  in every  $\beta_d$ -avoiding completion.

*In particular, the stack is determined by the scanned prefix: it consists of the unread values above  $b_d$ , cut exactly at the adjacent pairs that satisfy (ii).*

*Proof.* Since  $b_d = q$  by (17), (i) and (ii) are equivalent by Lemma 2.7. We prove that (ii) implies (iii) and that (iii) implies (i).

Suppose (ii) holds, and let  $c$  be the trigger. If  $v$  preceded  $u$  in a completion, an increasing  $d$ -subsequence ending at  $c$ , followed by  $x, v, u$ , would form  $\beta_d$ , since  $c < u < x < v$ . Therefore (iii) holds.

Suppose (i) fails, so that  $u$  and  $v$  lie in the same interval. The avoiding completion constructed in the proof of Proposition 2.8(c) reads each interval in decreasing order, and so reads  $v$  before  $u$ . Therefore (iii) fails.  $\square$

Condition (ii) locates the separations, not the letters that produced them: several letters between  $u$  and  $v$  may satisfy (ii), and the letter whose split created the separation need not be among them. Adjacency is needed only in (ii), as the example before Example 2.9 shows. For arbitrary unread  $u < v$  above  $b_d$ , conditions (i) and (iii) remain equivalent, and they hold exactly when some pair  $u' < v'$  of adjacent unread values with  $u \leq u' < v' \leq v$  satisfies (ii).

**Proposition 4.5.** *For fixed  $d \geq 1$  and  $n \geq d + 1$ , literal memoization of (20) has at least  $F_{n-d}$  distinct composition arguments when computing  $a_n^{(d)}$ , where  $F_{n-d}$  is the  $(n-d)$ th Fibonacci number,  $F_1 = F_2 = 1$ .*

*Proof.* Reading  $1, 2, \dots, d$  first reaches the state  $(\mathbf{0}, (n-d))$ : the first  $d-1$  letters are early-band moves, and  $d$  is a last-band move whose  $n-d$  larger unread values form the head. From there the interior choices in the proof of Proposition 2.13, with  $n$  replaced by  $n-d+1$ , reach  $F_{n-d}$  distinct compositions.  $\square$

The literal state space is therefore exponential in  $n$ . Section 5 removes the stack from the memoization key.

## 5. PROTECTED TAILS AND TRANSFER KERNELS

In this section we factor the literal recurrence through transfer kernels and derive recurrences for the kernels and for the empty-stack values. As shown after (21),  $H_{\mathbf{p}}(L)$  is the number of maximal transition sequences from  $(\mathbf{p}, L)$ , for every state. We mark an arbitrary tail  $L$  of the stack and define exposures, stopped paths and the identity kernel  $K_{\emptyset}$  as in Section 2. An early-band move changes only the control. A last-band move can enlarge the active head, and a letter of the active head replaces it by its parts below and above that letter. Thus, before  $L$  is exposed, the transitions change only the control and the blocks before the marker, and (20) shows that they and their targets are the same for every  $L$ . Every maximal transition sequence ends at the zero state, whose stack is empty. These are the two properties used in the proof of Theorem 2.14, and the same proof gives the following corollary. For a nonempty list  $U$  we let  $K_U^L(\mathbf{p}, \mathbf{t})$  be the number of stopped paths from  $(\mathbf{p}, U \mid L)$  to  $(\mathbf{t}, \emptyset \mid L)$ , and we write  $K_{\ell}^L = K_{\ell}^L$  and  $K_0 = K_{\emptyset}$ .

**Corollary 5.1** (Protected-tail factorization for  $\beta_d$ ). *The number  $K_U^L(\mathbf{p}, \mathbf{t})$  is independent of  $L$ . We write  $K_U(\mathbf{p}, \mathbf{t})$  for it. Then, for all lists  $U, V$  and  $L$  of positive integers,*

$$H_{\mathbf{p}}(UL) = \sum_{\mathbf{t} \in \mathbb{N}^d} K_U(\mathbf{p}, \mathbf{t}) H_{\mathbf{t}}(L) \quad \text{and} \quad K_{UV} = K_U K_V. \quad (23)$$

In particular, for  $L = (\ell_1, \dots, \ell_s)$ ,

$$H_{\mathbf{p}}(L) = \sum_{\mathbf{t} \in \mathbb{N}^d} (K_{\ell_1} K_{\ell_2} \cdots K_{\ell_s})(\mathbf{p}, \mathbf{t}) H_{\mathbf{t}}(\emptyset). \quad (24)$$

As in Section 2.6, we call  $K_{\ell}(\mathbf{p}, \cdot)$  a *kernel row*, with *source control*  $\mathbf{p}$  and *terminal controls*  $\mathbf{t}$ , and its support is the set of  $\mathbf{t}$  with  $K_{\ell}(\mathbf{p}, \mathbf{t}) \neq 0$ . The factorization replaces the composition argument by a product of transfer kernels, evaluated on the required source rows. The recurrence for the kernel rows follows, and Section 6 bounds the number of source and terminal controls required for the values  $|\text{Av}_n(\beta_d)|$  with  $n \leq N$ .

**Proposition 5.2** (Compressed kernel recurrence). *For  $\ell \geq 1$ , the transfer kernels satisfy*

$$\begin{aligned} K_{\ell}(\mathbf{p}, \mathbf{t}) &= \sum_{i=0}^{d-2} \sum_{h=0}^{p_i-1} K_{\ell}(T_{i,h}(\mathbf{p}), \mathbf{t}) \\ &\quad + \sum_{h=0}^{p_{d-1}-1} K_{\ell+\delta_h}(U_h(\mathbf{p}), \mathbf{t}) \\ &\quad + \sum_{\substack{a,b \geq 0 \\ a+b=\ell-1}} \sum_{\mathbf{u} \in \mathbb{N}^d} K_a(\mathbf{p}, \mathbf{u}) K_b(\mathbf{u}, \mathbf{t}). \end{aligned} \quad (25)$$

*Proof.* The first-move partition of the proof of [Proposition 2.16](#) applies, with [Corollary 5.1](#) in place of [Theorem 2.14](#). Early-band moves give the first sum, and a last-band move enlarges the head by  $\delta_h$ .  $\square$

We now connect the kernels to an empty stack. Put  $G_{\mathbf{p}} = H_{\mathbf{p}}(\emptyset)$ .

**Corollary 5.3** (Empty-stack recurrence). *The empty-stack values satisfy*

$$G_{\mathbf{p}} = \begin{cases} 1, & \mathbf{p} = \mathbf{0}, \\ \sum_{i=0}^{d-2} \sum_{h=0}^{p_i-1} G_{T_{i,h}(\mathbf{p})} + \sum_{h=0}^{p_{d-1}-1} \sum_{\mathbf{t} \in \mathbb{N}^d} K_{\delta_h}(U_h(\mathbf{p}), \mathbf{t}) G_{\mathbf{t}}, & \mathbf{p} \neq \mathbf{0}, \end{cases} \quad (26)$$

and

$$a_n^{(d)} = G_{(n,0,\dots,0)}. \quad (27)$$

*Proof.* This follows from (20) with an empty stack. A last-band move creates a head of size  $\delta_h$ , to which (23) applies, and  $K_0$  covers the case  $\delta_h = 0$ . The last equation is (22).  $\square$

## 6. THE FAMILY ALGORITHM AND ITS COMPLEXITY

In this section we bound the supports of the kernel rows and count the operations, stored integers and bit sizes of the resulting algorithm, which proves [Theorem 1.1](#). As in [Section 2.6](#), we give a kernel source  $(\mathbf{p}, \ell)$  the grade

$$w = \|\mathbf{p}\|_1 + \ell.$$

We verify below that increasing grade is a topological order for the kernel dependencies in (25), and that increasing control mass orders the empty-stack recurrence (26). The split term is controlled by the following support bound.

**Lemma 6.1** (Kernel support). *For every  $\mathbf{p} \in \mathbb{N}^d$  and  $\ell \geq 1$ ,*

$$\text{supp } K_{\ell}(\mathbf{p}, \cdot) \subseteq \{\mathbf{p}\} \cup \{\mathbf{t} : \|\mathbf{t}\|_1 \leq \|\mathbf{p}\|_1 - 1\}. \quad (28)$$

*Proof.* As in the proof of [Lemma 2.17](#), a base move strictly lowers the control mass and an active-head move leaves the control unchanged.  $\square$

In the algorithm below, each row operation is coefficientwise, and each row is accumulated from zero.

*Kernel phase.* For  $w = 1, 2, \dots, N$ , and every  $\mathbf{p} \in \mathbb{N}^d$ ,  $\ell \geq 1$  with  $\|\mathbf{p}\|_1 + \ell = w$ , evaluate (25) coefficientwise and store the resulting sparse row. In each split term, for every  $a, b \geq 0$  with  $a + b = \ell - 1$  and  $\mathbf{u} \in \text{supp } K_a(\mathbf{p}, \cdot)$ , add the scalar multiple  $K_a(\mathbf{p}, \mathbf{u})K_b(\mathbf{u}, \cdot)$  to the row being computed.

*Empty-stack phase.* Set  $G_{\mathbf{0}} = 1$ . For  $m = 1, 2, \dots, N$ , and every  $\mathbf{p} \in \mathbb{N}^d$  with  $\|\mathbf{p}\|_1 = m$ , evaluate (26), using  $K_0$  when  $\delta_h = 0$ . Return  $(G_{(n,0,\dots,0)})_{0 \leq n \leq N}$ .

As in the proof of [Theorem 2.19](#), with [Lemma 6.1](#) in place of [Lemma 2.17](#), increasing  $w$  is a topological order for the kernel phase and increasing  $m$  for the empty-stack phase. The new terms are the early-band rows of (25), of grade  $w - 1$  since  $\|T_{i,h}(\mathbf{p})\|_1 = \|\mathbf{p}\|_1 - 1$ , and the last-band rows, of grade  $w - 1$  since

$$\|U_h(\mathbf{p})\|_1 + (\ell + \delta_h) = \|\mathbf{p}\|_1 - (\delta_h + 1) + \ell + \delta_h = w - 1.$$

In (26), the early-band controls have mass  $m - 1$ , and by [Lemma 6.1](#) every terminal control  $\mathbf{t}$  of  $K_{\delta_h}(U_h(\mathbf{p}), \cdot)$  has  $\|\mathbf{t}\|_1 \leq \|U_h(\mathbf{p})\|_1 = m - 1 - \delta_h$ , trivially when  $\delta_h = 0$ . The kernel phase computes the rows of the *padded region*  $\|\mathbf{p}\|_1 + \ell \leq N$ ,  $\ell \geq 1$ , which by stars and bars has

$$\binom{N+d}{d+1} = O_d(N^{d+1}) \quad (29)$$

rows. By Lemma 6.1, a row has at most  $1 + (\|\mathbf{p}\|_1 + d - 1) = O_d(N^d)$  possible terminal controls.

*Proof of Theorem 1.1.* Correctness follows from Theorem 4.3, Corollaries 5.1 and 5.3, and Proposition 5.2: the algorithm evaluates (25) and (26) in the preceding topological orders and returns the values (27). We now bound the resources.

There are  $O_d(N^{d+1})$  source rows and  $O_d(N^d)$  possible entries per row, giving  $O_d(N^{2d+1})$  stored integers. In a split term there are  $O(N)$  choices of  $a$ ,  $O_d(N^d)$  intermediate controls  $\mathbf{u}$ , and  $O_d(N^d)$  entries in the row added for each  $\mathbf{u}$ . Thus a sparse implementation uses  $O_d(N^{2d+1})$  operations per source row and  $O_d(N^{3d+2})$  over all kernels. The nonsplit part uses  $O_d(N)$  row additions per source and therefore  $O_d(N^{2d+2})$  operations in total. For each of the  $O_d(N^d)$  empty-stack controls, (26) sums at most  $O(N)$  choices over  $O_d(N^d)$  terminal controls, so the  $G$  table costs  $O_d(N^{2d+1})$ . For  $d \geq 1$ , the exponents  $2d + 2$  and  $2d + 1$  are both smaller than  $3d + 2$ .

By the argument in the last paragraph of the proof of Theorem 2.19, where a state now has at most  $\rho(\mathbf{p}, L) \leq N$  unread values and as many legal next letters, every stored integer and every intermediate product and partial sum is at most  $(N + 1)^N$  and has  $O(N \log N)$  bits. Replacing integer multiplication by its  $M(N \log N)$  bit cost proves the stated bit complexity. The bit storage is  $O_d(N^{2d+2} \log N)$ .  $\square$

## 7. THE TWO-THRESHOLD QUOTIENT FOR 12453

In this section we specialize to  $d = 2$  and write  $\mathbf{p} = (p, q)$ , so that  $q$  is the size of band 1, not the least 2-trigger of (17). Two symmetries of the kernels are available in this case, and they serve different purposes. The first is a translation in the first control coordinate: a kernel entry depends on the first coordinates of its source and terminal controls only through their difference, so one coordinate can be dropped from the table. This is the symmetry that improves the  $d = 2$  specialization  $O(N^8)$  and  $O(N^5)$  of Theorem 1.1 to the  $O(N^7)$  and  $O(N^4)$  of Corollary 7.4. The second is a partial translation in the second control coordinate. It leaves those exponents unchanged and halves the leading constant in the storage count. We first determine the support of a kernel row exactly, which refines the bound of Lemma 6.1.

**Lemma 7.1** (Exact two-threshold support). *For every  $\ell \geq 1$ ,*

$$\text{supp } K_\ell((p, q), \cdot) = \{(p, q)\} \cup \{(u, v) : 0 \leq u \leq p, v \geq 0, u + v \leq p + q - 1\}. \quad (30)$$

*Proof.* A base move never increases the first coordinate and strictly lowers the coordinate sum, which proves the inclusion. The terminal control  $(p, q)$  is reached by using no base move and reading endpoints until the head is exhausted. To reach a control  $(u, v)$  in the second set with  $u < p$ , first make the early-band move with  $h = u$ , which gives  $(u, Q)$  with  $Q = p + q - 1 - u$ . The support inequality says  $v \leq Q$ . If  $v = Q$ , make no further base move. If  $v < Q$ , make the last-band move with  $h = v$ . If  $u = p$ , then  $v \leq q - 1$ , and the last-band move with  $h = v$  can be made directly. Finally, exhaust the active head by endpoint choices, which exposes the marked suffix at  $(u, v)$ .  $\square$

We now prove the first translation.

**Lemma 7.2** (First-coordinate translation). *For  $\ell \geq 1$  and  $0 \leq c \leq p$ ,*

$$K_\ell((p, q), (c, s)) = K_\ell((p - c, q), (0, s)). \quad (31)$$

*Proof.* No transition increases the first control coordinate: an early-band move with parameter  $h$  lowers it from  $p'$  to  $h$ , and the other moves keep it. Every state of a stopped path from  $((p, q), (\ell) \mid L)$  to  $((c, s), \emptyset \mid L)$  therefore has first coordinate at least  $c$ , and every early-band move on it has parameter  $h \geq c$ . Lower every first coordinate and every early-band parameter on the path by  $c$ . By (18), an early-band move from  $(p', q')$  with parameter  $h$  becomes the move from

$(p' - c, q')$  with parameter  $h - c$ , whose target  $(h - c, q' + p' - 1 - h)$  is the old target with its first coordinate lowered by  $c$ . By (19), a last-band move keeps its parameter, the second coordinate of its target and its head enlargement, and active-head moves do not involve the control. The result is a stopped path from  $((p - c, q), (\ell) \mid L)$  to  $((0, s), \emptyset \mid L)$ , and raising every first coordinate and every early-band parameter by  $c$  is the inverse map.  $\square$

The proof uses only two facts: no transition increases the first control coordinate, and, by (18) and (19), if the first coordinate of the source of a move is lowered by  $c$ , and its parameter  $h$  also when the move reads a value of  $B_0$ , then the target changes only in its first coordinate, which is lowered by  $c$ , and the head enlargement is unchanged. Both facts hold for every  $d$ , so that

$$K_\ell(\mathbf{p}, \mathbf{t}) = K_\ell(\mathbf{p} - t_0 \mathbf{e}_0, \mathbf{t} - t_0 \mathbf{e}_0) \quad \text{whenever } t_0 \leq p_0,$$

where  $\mathbf{e}_0 = (1, 0, \dots, 0)$ . Storing only the entries with  $t_0 = 0$  and counting as in the proofs of Theorem 1.1 and Corollary 7.4 gives  $O_d(N^{3d+1})$  operations and  $O_d(N^{2d})$  stored integers, that is,  $O(N^4)$  and  $O(N^2)$  for 1342, and the bounds of Corollary 7.4 for 12453. We have stated Theorem 1.1 in the simpler form.

We write

$$R_{\ell,a}(q, s) := K_\ell((a, q), (0, s)). \quad (32)$$

We call  $R_{\ell,a}(q, s)$  a *reduced entry* and the table of all such entries the *first-coordinate quotient*. Throughout this section the letter  $a$  denotes a reduced first control coordinate, not a block size as in (5) and (25). Block sizes in a split are written  $\ell_1, \ell_2$ . By Lemma 7.2, every kernel entry is represented by one such reduced entry, with  $a = p - c$  (entries with  $c > p$  vanish by Lemma 7.1). By Lemma 7.1, for a reduced source  $(\ell, a, q)$  with  $a = 0$  the possible terminal coordinates are  $0 \leq s \leq q$ , and if  $a > 0$  a base move is necessary and  $0 \leq s \leq a + q - 1$ . The first-coordinate quotient therefore has

$$\binom{N+2}{3} \text{ rows, } \quad \frac{N(N+1)(N^2+N+4)}{12} \text{ entries.} \quad (33)$$

The entry formula follows by summing  $q+1$  for  $a=0$  and  $a+q$  for  $a>0$  over the same padded source region. In particular, it already gives  $O(N^4)$  storage. We count the operations in the proof of Corollary 7.4.

We now rewrite the kernel recurrence in the reduced variables by applying Lemma 7.2 to (25). An early-band move from  $(a, q)$  gives  $R_{\ell,h}(a+q-h-1, s)$ , and a last-band move with parameter  $r$ , written  $r$  here because  $h$  indexes the early-band sum, gives  $R_{\ell+q-r-1,a}(r, s)$ . In a split with intermediate control  $(c, m)$ , the first factor  $K_{\ell_1}((a, q), (c, m))$  vanishes unless  $c \leq a$ , since both parts of the support (30) have first coordinate at most  $a$ . For  $0 \leq c \leq a$  the two factors become

$$R_{\ell_1,a-c}(q, m) R_{\ell_2,c}(m, s).$$

The split terms of (25) with a factor  $K_0$  contribute  $\mathbf{1}_{\mathbf{p}=\mathbf{t}}$  when  $\ell = 1$  and  $2K_{\ell-1}(\mathbf{p}, \mathbf{t})$  when  $\ell \geq 2$ , and we collect them into a term  $D_{\ell,a}$ . Writing  $a_1 = a - c$  and  $a_2 = c$  therefore gives the reduced recurrence

$$\begin{aligned} R_{\ell,a}(q, s) = & \sum_{h=0}^{a-1} R_{\ell,h}(a+q-h-1, s) + \sum_{r=0}^{q-1} R_{\ell+q-r-1,a}(r, s) + D_{\ell,a}(q, s) \\ & + \sum_{\substack{\ell_1, \ell_2 \geq 1 \\ \ell_1 + \ell_2 = \ell - 1}} \sum_{\substack{a_1, a_2 \geq 0 \\ a_1 + a_2 = a}} \sum_{m \geq 0} R_{\ell_1, a_1}(q, m) R_{\ell_2, a_2}(m, s), \end{aligned} \quad (34)$$

where

$$D_{1,a}(q, s) = \mathbf{1}_{a=0, q=s}, \quad D_{\ell,a}(q, s) = 2R_{\ell-1,a}(q, s) \quad (\ell \geq 2),$$

and entries outside the support above are zero.

We call  $\ell + a + q$  the *reduced grade* of  $R_{\ell,a}(q, \cdot)$ . Increasing reduced grade is a topological order for (34). Every term of the first two sums and every  $D$  term has reduced grade  $\ell + a + q - 1$ . In a split, the first factor has reduced grade  $\ell_1 + a_1 + q \leq \ell + a + q - 2$ . A nonzero first factor forces  $m \leq q$  when  $a_1 = 0$  and  $m \leq a_1 + q - 1$  when  $a_1 > 0$ , so the second factor has reduced grade  $\ell_2 + a_2 + m \leq \ell + a + q - 2$ .

The second control coordinate admits a partial quotient as well: raising both the second source coordinate and the terminal coordinate by one leaves a reduced entry unchanged whenever the terminal coordinate is at least  $a - 1$ .

**Lemma 7.3** (Second-coordinate translation). *For  $\ell \geq 1$  and  $a, q, s \geq 0$ , whenever  $s \geq a - 1$ ,*

$$R_{\ell,a}(q + 1, s + 1) = R_{\ell,a}(q, s). \quad (35)$$

*Proof.* We label the unread values. In a state with stack  $U \mid L$ , give the values of band 0, of band 1, and of the intervals of  $U$  distinct labels, ordered as the values are. The transitions of (20) from such a state correspond to the labels of the two bands and of the active head. An early-band move with parameter  $h$  reads the label of band 0 with  $h$  labels of band 0 below it, and the labels of band 0 above it join band 1, below its labels. A last-band move with parameter  $h$  reads the label of band 1 with  $h$  labels of band 1 below it, and the labels of band 1 above it join the active head, below its labels. A letter of the active head of local rank  $r$  reads the head label with  $r - 1$  head labels below it, and the head is replaced by its labels below and above that label, in this order. So a stopped path from a labeled state is determined by the sequence of labels it reads.

Consider a stopped path counted by  $R_{\ell,a}(q + 1, s + 1)$ , and let  $z$  be the least label of the initial band 1. Suppose that  $z$  leaves band 1, by being read or by joining the head. This happens in a last-band move that reads  $z$  or a label below it, and every label of the initial band 1 still in band 1 joins the head or is read in the same move. Afterwards labels enter band 1 only from band 0, so the  $s + 1$  labels of the terminal band 1 belong to the initial band 0. Band 0 loses labels only through early-band moves, each of which reads one of its labels, and it ends empty. So at most  $\max(a - 1, 0)$  labels of the initial band 0 remain unread, while  $s + 1 \geq \max(a, 1)$  since  $s \geq a - 1$ . This contradiction shows that  $z$  stays unread in band 1. In particular every last-band move reads a label above  $z$ . Deleting  $z$  from every state leaves the early-band parameters and the local ranks of head letters unchanged, and lowers each last-band parameter and the size of band 1 by one, which leaves each head enlargement  $\delta_h$  unchanged. Since  $z$  never enters the stack, the stacks are unchanged and  $L$  is exposed at the same step. This gives a stopped path counted by  $R_{\ell,a}(q, s)$  that reads the same labels.

Conversely, consider a stopped path counted by  $R_{\ell,a}(q, s)$ . No last-band move of it reads a label  $y$  of the initial band 0. Such a  $y$  would have entered band 1 through an early-band move, which read another label of the initial band 0, and reading  $y$  would send every label of the initial band 1 still in band 1 into the head. The terminal band 1 would then consist of at most  $a - 2$  labels of the initial band 0, contradicting  $s \geq a - 1$ . So every last-band move reads a label of the initial band 1. Insert a label  $z$  into the initial band 1 below its labels. The same labels can then be read, with each last-band parameter and the size of band 1 raised by one and the head enlargements unchanged. Since every last-band move reads a label above  $z$ , the label  $z$  stays unread in band 1, so the stacks are unchanged and  $L$  is exposed at the same step. This gives a stopped path counted by  $R_{\ell,a}(q + 1, s + 1)$ . The two operations are inverse.  $\square$

The hypothesis  $s \geq a - 1$  cannot be weakened: equation (34) gives  $R_{1,2}(1, 1) = 3$  but  $R_{1,2}(0, 0) = 2$ .

For  $a = 0$  and  $0 \leq s \leq q$ , Lemma 7.3 gives  $R_{\ell,0}(q, s) = R_{\ell,0}(q - s, 0)$ . For  $a \geq 1$  and  $s \geq a$ , it gives

$$R_{\ell,a}(q, s) = R_{\ell,a}(q - s + a - 1, a - 1),$$

where the support condition ensures that the new second source coordinate is nonnegative. Thus we retain only terminal coordinate  $s = 0$  when  $a = 0$ , and terminal coordinates  $0 \leq s < a$  when  $a \geq 1$ . The exact number of retained entries is

$$\binom{N+1}{2} + \binom{N+2}{4} = \frac{N(N+1)(N^2+N+10)}{24}. \quad (36)$$

Thus the second translation halves the leading storage constant relative to the first-coordinate quotient, and each omitted entry is recovered from (35) in  $O(1)$  operations. Exact polynomials for loop iterations and stored entries are recorded in the computation report `notes/av12453_150_computation_report.md` of the supplement.

**Corollary 7.4** (Sharpened 12453 bound). *For every  $N \geq 1$ , the numbers  $|\text{Av}_n(12453)|$  for  $0 \leq n \leq N$  can be computed using  $O(N^7)$  integer arithmetic operations and  $O(N^4)$  stored integers. The bit complexity is  $O(N^7 M(N \log N))$ , and the bit storage is  $O(N^5 \log N)$ , with  $M$  as in Theorem 1.1.*

*Proof.* The algorithm evaluates (34) for the reduced sources  $(\ell, a, q)$  with  $\ell + a + q \leq N$  in increasing reduced grade, which the preceding paragraphs showed to be a topological order, then evaluates (26), reading every kernel entry from the first-coordinate quotient by (31), and returns the values (27). It is correct by the proof of Theorem 1.1 and Lemmas 7.1 and 7.2. We count its operations. The first-coordinate quotient has  $O(N^3)$  source rows and  $O(N^4)$  stored entries by (33). Evaluating the split terms involves  $O(N)$  choices for the block split  $\ell_1$ ,  $O(N)$  choices for the control split  $a_1$ , and  $O(N)$  possible intermediate second coordinates  $m$ . For each such triple, adding the row  $R_{\ell_2, a_2}(m, \cdot)$  uses  $O(N)$  possible terminal coordinates. Thus one source row costs  $O(N^4)$  operations and all split terms cost  $O(N^7)$ . Evaluating the nonsplit kernel terms costs at most  $O(N^5)$ . The empty-stack table has  $O(N^2)$  source controls,  $O(N)$  last-band choices, and  $O(N^2)$  possible terminal controls, so it also costs  $O(N^5)$ . The bit-size argument in the proof of Theorem 1.1 gives  $O(N \log N)$  bits per integer, which gives the stated bit bounds.  $\square$

## 8. THE COUNTING SEQUENCE OF $\text{Av}(12453)$

In this section we describe the computation of  $|\text{Av}_n(12453)|$  for  $n \leq 150$  and, in Section 8.1, uniform random generation. Table 1 lists some of the values. The file `code/data/av12453_terms_0_150.txt` in the supplement contains all of them.

We evaluated the reduced recurrence modulo pairwise distinct primes and reconstructed the values over the integers. The programs `av12453_fast_rns.cpp` and `reconstruct_av12453_rns.py` in the supplement perform the modular evaluation, the check of the coefficient bound, and the reconstruction.

The reconstruction rests on a bound for the coefficients. With  $a_m^{(1)} = |\text{Av}_m(1342)|$  as in (22), put

$$\mathcal{B}_N := \sum_{m=0}^N \binom{N}{m}^2 a_m^{(1)}. \quad (37)$$

As Bóna [11] observed, deleting the left-to-right minima of a 12453-avoider leaves a 1342-avoider: the first entry of an occurrence of 1342 among the remaining entries is not a left-to-right minimum, so an earlier smaller entry precedes it and forms a 12453 with the occurrence. If  $m$  entries remain, choose their positions, their values, and their standardization. The deleted minima occupy the remaining positions and take the remaining values in decreasing order. Therefore  $|\text{Av}_N(12453)| \leq \mathcal{B}_N$ .

The integers  $a_m^{(1)}$  are computed from (15). The source `av12453_fast_rns.cpp` implements (34) and the empty-stack recurrence (26) modulo each selected prime. By Theorem 4.3, Corollaries 5.1

| $n$      | $ \text{Av}_n(12453) $                                                                                                                                                  |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0        | 1                                                                                                                                                                       |
| 1        | 1                                                                                                                                                                       |
| 2        | 2                                                                                                                                                                       |
| 3        | 6                                                                                                                                                                       |
| 4        | 24                                                                                                                                                                      |
| 5        | 119                                                                                                                                                                     |
| 6        | 694                                                                                                                                                                     |
| 7        | 4581                                                                                                                                                                    |
| 8        | 33286                                                                                                                                                                   |
| 9        | 260927                                                                                                                                                                  |
| $\vdots$ | $\vdots$                                                                                                                                                                |
| 70       | 272047014426505848053184970016495041863317183438034114760726065251824322                                                                                                |
| 71       | 3650982161839069245244704534509742637654076726349158226492884931736173219                                                                                               |
| 72       | 49052479122475377086437757538549643339461005664517035047738861064660650462                                                                                              |
| 73       | 659758130045267152279725187963519417028655461010134067851632959874299139314                                                                                             |
| 74       | 8883183490302999538631811574792997430948698470130245020946152645591389765992                                                                                            |
| 75       | 119729358528034563299140115867589732584037184044497744461031132334923227197902                                                                                          |
| 76       | 1615360029787371109415151060029379769950586589470709389115337666950300899664892                                                                                         |
| 77       | 21815422062572671915613939537695855501209448778205442702933011769377618433242686                                                                                        |
| 78       | 294898723790650390962604916545121449539951614378915249501191039739041483852959800                                                                                       |
| 79       | 3990128916983023825344047432227024955428537983460787346271686479611600994645296610                                                                                      |
| 80       | 54037578151511512886697895933429730101506005764295760914601440348390527553113513544                                                                                     |
| $\vdots$ | $\vdots$                                                                                                                                                                |
| 140      | 1325631292096789000970599917299726854851622293985609301526390639958435284778009278831298106<br>90309462018830746612767544034249954911304218723154384122701096           |
| 141      | 185288351260158636665154188074881299733908503673301584716105567137519163453156092505084472<br>656884441044425731642146730167064793041572656010317336745260230           |
| 142      | 259061502229303438137253209397747821095567080423357798067579823149354198936311384115189454<br>8670967420193329587946042347831801662279707120672955517288773276          |
| 143      | 362314147509666008769192778238021298134479862253752577900601633529337398739612132874209755<br>00425725343682999003885891306094188761363063724056324785823177873         |
| 144      | 506866570108080992172827702390824721485314592184236068629394550709633572784507558031741140<br>313981790054699490480890332852463168535779082217300140047632872622        |
| 145      | 709293982764754919593192205776244504591725854558410034064407257992516450166543645900069113<br>833616956508257221756849768300959452695409003062226217099351149132        |
| 146      | 992845185957788995426922964104645694631352710528923383764487517996963854569544958714672498<br>70870043899715835749168084058967962235009591429889624809811996930400      |
| 147      | 13901376928245085643377786818058778546882793194476967952737658048244365801756110716322666<br>2022575277081761365776749741699180828349495930184915117643473649518426     |
| 148      | 194694442948917259391570834796944420652677476522578329510621571835317403885469519572114751<br>35962040032085429461259232291433428857567215495002695620164810628780672   |
| 149      | 272751533859863976347598705423927955482798527767963594985771513751414865658702668306436986<br>953076260565473100844770203223196420613511632988251274688398628136581935  |
| 150      | 382205773917840409900771979163202054515626683096917522113147052334208885924065532828958769<br>7088297741511203960758500758477699391695509589920898592073159171767612574 |

TABLE 1. The numbers  $|\text{Av}_n(12453)|$  for  $0 \leq n \leq 9$ ,  $70 \leq n \leq 80$ , and  $140 \leq n \leq 150$ .

and 5.3, Proposition 5.2, and Lemmas 7.1 to 7.3, these recurrences give the coefficients, so a correct evaluation outputs their residues, and the checks below test the implementation. The moduli are the twenty-four largest primes below  $2^{31}$ , recorded with the residue packs in the supplement. The program `reconstruct_av12453_rns.py` verifies that the product  $\mathcal{M}$  of the eighteen largest exceeds  $\mathcal{B}_{150}$ . The margin is  $\mathcal{M}/\mathcal{B}_{150} > 1.92$ . Since the nonnegative bound  $\mathcal{B}_N$  is nondecreasing, every coefficient through degree 150 lies in  $[0, \mathcal{M})$ . The Chinese remainder theorem therefore determines each coefficient uniquely.

The six remaining primes, withheld from the reconstruction, agree with every reconstructed coefficient. A separately written single-modulus  $d = 2$  program computes all 151 coefficients modulo  $2^{61} - 1$  and also agrees. These are error-detection checks, not inputs to the uniqueness argument. The values through  $n = 38$  agree with those previously known, in OEIS A116485 [21], which come from Biers-Ariel's program [8].

**8.1. Uniform random generation.** The transfer tables can also be used to generate uniformly random avoiders. We call a maximal transition sequence a *complete path*. By [Theorem 4.3](#), the complete paths from the initial state  $((n, 0, \dots, 0), \emptyset)$  correspond bijectively to the  $\beta_d$ -avoiding permutations of length  $n$ . By the recursive method of Nijenhuis and Wilf [20], such a path is uniformly random when every transition is chosen with probability proportional to the number of complete paths through it, that is, to the value of the corresponding summand of (20). That value is a completion count  $H_{\mathbf{p}}(L)$  for a stack  $L$  of arbitrary length, which the factorization (24) expresses through the stored kernels. We therefore sample a complete path of the factorized recurrence itself. The first-move partitions in the proofs of [Proposition 5.2](#) and [Corollary 5.3](#) identify the summands of (25) and (26) with a partition of the stopped and complete paths. Iterating the cut-and-concatenate bijection of [Theorem 2.14](#) maps these paths onto the complete paths of (20). At an empty-stack state, a summand of (26) is chosen with probability proportional to its value. This selects a base letter and, for a last-band move, the terminal control  $\mathbf{t}$  at which the new head will have been exhausted. Inside a kernel  $K_\ell(\mathbf{p}, \mathbf{t})$ , a summand of (25) is chosen in the same way. This selects a base move, an endpoint of the active head, or an interior letter of it together with the intermediate control  $\mathbf{u}$  of the split, and each block of the stack carries the control at which it must be exposed. By induction on the grade and the control mass, a given stopped path counted by  $K_\ell(\mathbf{p}, \mathbf{t})$  is produced with probability  $1/K_\ell(\mathbf{p}, \mathbf{t})$ , and a given complete path from the state  $(\mathbf{p}, \emptyset)$  with probability  $1/G_{\mathbf{p}}$ . At  $\mathbf{p} = (n, 0, \dots, 0)$  this is  $1/|\text{Av}_n(\beta_d)|$ .

**Proposition 8.1** (Uniform random generation). *Let  $N \geq n \geq 0$ . Once the first-coordinate quotient  $R_{\ell,a}(q, s)$  of [Section 7](#) and the empty-stack values  $G_{(p,q)}$  have been computed for size  $N$ , a uniformly random element of  $\text{Av}_n(12453)$  can be produced with  $O(n^4)$  arithmetic operations on the stored integers.*

*Proof.* The paragraph before the proposition shows that the distribution is uniform. A permutation of length  $n$  is produced by  $n$  letter choices. Each choice scans the summands of one instance of (26) or (25) in a fixed order and stops at the first summand whose running total exceeds a uniformly random integer  $y$  with  $0 \leq y < Y$ , where  $Y$  is the stored value of the state. Every summand is one stored entry or a product of two, by [Lemma 7.2](#). Every kernel row consulted has reduced grade at most  $n$ , and every empty-stack control has mass at most  $n$ . The sampler keeps the value sets of the bands and of the intervals of the stack as sorted lists, so that it outputs the letter of each chosen transition and updates the sets in  $O(n)$  operations. At an empty-stack state the last-band terms range over the band position and the terminal control,  $O(n^3)$  summands. Inside a kernel the interior terms range over the split position and the intermediate control, again  $O(n^3)$ . The other groups are smaller. Therefore each choice costs  $O(n^3)$  operations and a sample costs  $O(n^4)$ .  $\square$

Unlike the coefficients of [Table 1](#), our sampler runs in floating point. [Proposition 8.1](#) is the exact-arithmetic statement. [Appendix A](#) describes the implementation and proves that, for  $n \leq 300$  and with ideal random bits, its output is within total variation distance  $3.5 \cdot 10^{-5}$  of the uniform distribution on  $\text{Av}_n(12453)$  ([Proposition A.2](#)). Every sample is verified to avoid 12453 by an independent test based on [Lemma 2.1](#), and the sampler was checked against exhaustive enumeration for  $7 \leq n \leq 10$ . Building the tables for  $N = 300$  took about four hours on an

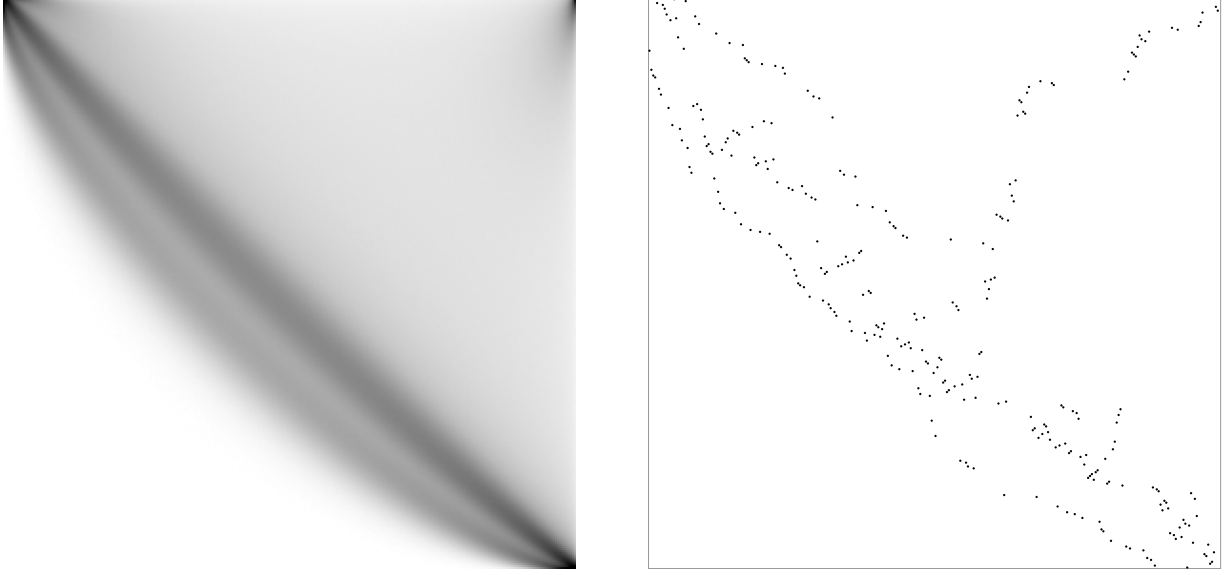


FIGURE 4. Left: position–value heatmap of one million elements of  $\text{Av}_{300}(12453)$  drawn by the floating-point sampler. The cell at position  $i$  (left to right) and value  $v$  (bottom to top) is shaded by the square root of the number of samples with  $\pi_i = v$ , from white (no sample) to black (the largest cell count, 70 204). Right: one of the samples, with a dot at  $(i, \pi_i)$  in the same orientation. Its first letter is 272 and it has 71 left-to-right minima.

eight-core aarch64 machine and produced a table of 5.4 GB, which the sampler holds in memory. One million samples of length 300 then took about thirty-five minutes.

Figure 4 shows the resulting position–value heatmap, rendered with the script that draws the heatmaps of the PermPAL database [2, 1], beside one of the samples. The first letter has mean 291.9 over the one million samples. The dark band from the top left to the bottom right shows that a typical avoider stays close to a decreasing curve. The left-to-right minima lie below it, and their mean number is 77.5. The single sample shows the same features: a decreasing band of letters with the left-to-right minima scattered below it, and above it the letters exceeding the first letter, which form a 1342-avoider.

## 9. FORMAL VERIFICATION

The literal and kernel recurrences of this paper for  $d \leq 2$  have been verified in the Lean 4 proof assistant [14] with the Mathlib library [24]. The development is in the directory `formal/` of the supplement. Its four main theorems state that, for every  $n$ ,

$$\begin{aligned} W_n(\emptyset) &= |\text{Av}_n(1342)|, & G_n &= |\text{Av}_n(1342)|, \\ H_{(n,0)}(\emptyset) &= |\text{Av}_n(12453)|, & G_{(n,0)} &= |\text{Av}_n(12453)|. \end{aligned}$$

The development proves the counting statement of Theorem 4.3 for  $d \leq 2$ , that a legal prefix with state  $(\mathbf{p}, L)$  has  $H_{\mathbf{p}}(L)$  completions to a  $\beta_d$ -avoiding permutation, and the first and third theorems are its case of the empty prefix. The second and fourth say that the recurrences evaluated by the algorithm of Section 6 count  $\text{Av}_n(1342)$  and  $\text{Av}_n(12453)$ .

A reader who wants to verify this needs to do two things. The first is to run the scripts `check.sh` and `comparator.sh`. The script `check.sh` builds the development and checks that its proof libraries and the file `Solution.lean` contain no unproved statement and that every theorem

of the libraries depends only on Lean’s three standard axioms. The script `comparator.sh` checks, with Lean’s kernel and the two independent kernels `nanoda` and `con-ron`, that the proofs in `Solution.lean` prove exactly the statements of the file `Challenge.lean` and use no other axioms. In `Challenge.lean` each of the four statements has the placeholder `sorry` in place of its proof. The second thing to do is to read `Challenge.lean`, about 180 lines that import only `Mathlib` and contain the four statements together with every definition they use: pattern containment, the patterns 1342 and 12453, and the recurrences. The proofs need not be read.

The proofs follow the paper. The interval stack of a scanned prefix is computed from the prefix by the moves of [Proposition 4.1\(b\)](#) and its control from the thresholds, the separation criterion of [Lemma 2.7](#) is the inductive invariant, [Proposition 2.8\(a\)](#) and (b) are derived from it, and the recurrences follow by induction on the number of unread values. The kernels are defined by their recurrences, and the factorization (23) for a single head  $U = (\ell)$  is proved from the equations of  $H$  by induction on the grade rather than by cutting paths. Not formalized are, among others, the complexity statements of [Theorem 1.1](#) and [Corollary 7.4](#); the description of the kernels as counts of stopped paths; the translation quotients of [Section 7](#) and the bound  $|\text{Av}_N(12453)| \leq \mathcal{B}_N$ , and with them the programs of [Section 8](#); [Proposition 8.1](#) and [Appendix A](#); condition (iii) of [Corollary 4.4](#); [Proposition 4.5](#); and the family for  $d \geq 3$ . [Appendix B](#) describes the development in more detail.

## 10. CONSEQUENCES AND OUTLOOK

In this section we place 12453 among the length-five classes and state a conjecture on the asymptotics of its counting sequence. Two patterns are *Wilf-equivalent* if they have equally many avoiders of every length. Reverse, complement, and inverse preserve all counting sequences. The dihedral orbit of 12453 has eight elements, and the prefix-reversal equivalence of Backelin–West–Xin [7] gives  $12453 = 12 \oplus 231 \sim 21 \oplus 231 = 21453$ , whose orbit supplies eight more. Therefore the algorithm, bounds, and coefficients apply to all sixteen members of the length-five Wilf class represented by 12453, which has no other members [13, Table 1].

Bóna determined the Stanley–Wilf limit of every member of the family [11]:

$$\lim_{n \rightarrow \infty} (a_n^{(d)})^{1/n} = (d - 1 + \sqrt{8})^2.$$

For the remainder of this section, put  $a_n = a_n^{(2)} = |\text{Av}_n(12453)|$  and  $\mu = (1 + \sqrt{8})^2 = 9 + 4\sqrt{2}$ . The coefficients through  $n = 100$ , read with the series analysis of Clisby, Conway, Guttmann, and Inoue [13], suggest the following refinement.

**Conjecture 10.1.** *There are constants  $C, \kappa > 0$  and  $g, h \in \mathbb{R}$  such that, as  $n \rightarrow \infty$ ,*

$$a_n = C\mu^n e^{-\kappa n^{1/3}} n^g \left(1 + hn^{-1/3} + O(n^{-2/3})\right). \quad (38)$$

The stretched exponent  $1/3$  agrees with the value that the 2022 analysis found most likely [13, Section 10.3]. Unweighted least squares on  $70 \leq n \leq 100$  for the model

$$\log a_n = n \log \mu - \kappa n^{1/3} + g \log n + \log C + hn^{-1/3}$$

gives  $g \approx -4.251$ , and we fixed the nearby value  $g = -17/4$ . Both exponents were chosen using only  $a_0, \dots, a_{100}$ . With  $g = -17/4$ , the same fit gave the parameters  $\kappa, \log C, h$ , which were fixed before  $a_{101}, \dots, a_{150}$  were computed. The frozen values are  $\kappa = 1.34550865$ ,  $C = 0.82710184$  and  $h = 1.36800770$ , to the digits shown. On those fifty excluded coefficients, the maximum absolute difference between the observed and predicted values of  $\log a_n$  is below  $4.3 \cdot 10^{-7}$ . The two fits and this check are reproduced by a program in the supplement.

From shorter series, the 2022 analysis concluded that the stretched-exponential base  $e^{-\kappa}$  (their  $\mu_1$ ) is  $0.27 \pm 0.02$  and that the power-law exponent is  $-4.7 \pm 0.3$  [13, Section 10.3]. The frozen fit

gives  $e^{-\kappa} \approx 0.260$ , inside that range. Its exponent  $-17/4$  lies outside the 2022 range, but we do not regard the two as in conflict. On  $70 \leq n \leq 150$  the exponent  $-17/4$  fits the model with one correction term far better than their single-fit value  $-4.67$  does. The preference shrinks by more than an order of magnitude once a second correction term  $n^{-2/3}$  is admitted. In fits with two correction terms, a change in the stretched exponent  $1/3$  is largely compensated by changes in the power-law exponent and in  $\kappa$ . The inferred power-law exponent is therefore sensitive to the correction terms included, and the available coefficients do not determine it reliably, which is why [Conjecture 10.1](#) leaves  $g$  unspecified. We report  $g \approx -17/4$  as the value fitted with the stretched exponent  $1/3$  and one correction term, not as a sharpening of the 2022 estimate. The holdout test supports [Conjecture 10.1](#) but does not prove it.

#### DATA AND CODE AVAILABILITY

The code, coefficients, residues, reconstruction script, and computation reports used for [Section 8](#), the sampler and heatmap tools of [Section 8.1](#), the Permuta walkthrough with its exhaustive checks for length at most 8, and the Lean development of [Section 9](#) and [Appendix B](#) are in the public repository

<https://github.com/ulfarsson/public-12453>

in its release v1.4.0, which is the supplement referred to throughout the paper. The code, data, and Lean development are released under the Apache License 2.0. The heatmap of [Figure 4](#) was drawn from the count matrix in the repository by the heatmap script of the PermPAL database, written by Jay Pantone and Vince Vatter, which is not part of the repository. The repository’s MANIFEST.sha256 records the SHA-256 digest of every file, and its README files give verification, reconstruction, and audit commands.

#### STATEMENT ON THE USE OF AI

Large language model assistants (Claude Opus and Claude Fable from Anthropic, and GPT Sol from OpenAI) were used throughout the preparation of this paper under the direction of the author: to draft and revise the text, to write and test the companion programs and run the computations reported in [Section 8](#), and to develop the Lean formalization of [Section 9](#). The mathematical content was specified and checked by the author. The computational claims are supported by the independent implementations and checks described in [Section 8](#) and, for the sampler, by the error bound of [Appendix A](#), and the statements of [Section 9](#) are verified by the Lean kernel. In addition, the Permuta walkthrough in the supplement checks every combinatorial statement of the paper that has finite content exhaustively for all permutations of length at most 8, by comparing the paper’s constructions and displayed formulas with brute-force computations from the definitions. These checks are written with the Permuta library [5], whose core code predates the large language model assistants, and they do not use the implementations described in [Section 8](#). The author takes full responsibility for the paper.

#### APPENDIX A. FLOATING-POINT SAMPLING

This appendix describes the floating-point implementation of the sampler of [Proposition 8.1](#) and bounds the distance of its output distribution from uniformity. Throughout, the tables have size  $N = 300$  and  $n \leq N$ .

**A.1. The implementation.** For  $d = 2$  and a control  $(p, q) \neq (0, 0)$ , the empty-stack recurrence (26) reads, in the reduced entries,

$$G_{(p,q)} = \sum_{h=0}^{p-1} G_{(h,p+q-1-h)} + \mathbf{1}_{q \geq 1} G_{(p,q-1)} + \sum_{h=0}^{q-2} \sum_{c=0}^p \sum_{t \geq 0} R_{q-1-h,p-c}(h, t) G_{(c,t)}. \quad (39)$$

The middle term is the last-band move with  $\delta_h = 0$ . At  $N = 300$  the largest values are about  $2^{1115}$ , beyond the largest binary64 number, which is below  $2^{1024}$ . The implementation therefore stores the scaled values

$$R'_{\ell,a}(q, s) = 2^{-2(\ell+a+q)} R_{\ell,a}(q, s), \quad G'_{(p,q)} = 2^{-2(p+q)} G_{(p,q)}.$$

In the scaled values, every band term and every  $D_{\ell,a}$  with  $\ell \geq 2$  in (34) is multiplied by  $1/4$ , the split term becomes  $4^{m-1} R'_{\ell_1,a_1}(q, m) R'_{\ell_2,a_2}(m, s)$ , and  $D_{1,a}(q, s)$  becomes  $2^{-2(1+a+q)} \mathbf{1}_{a=0, q=s}$ . In (39) the first two terms are multiplied by  $1/4$  and the last becomes  $4^{t-1} R'_{q-1-h,p-c}(h, t) G'_{(c,t)}$ . In the build for  $N = 300$  the nonzero scaled values lie between  $2^{-600}$  and  $2^{515}$ , inside the normal range of binary64, and none is infinite or subnormal.

A term  $4^{m-1} r_1 r_2$  with scaled factors  $r_1, r_2$  is formed as  $(2^{m-1} r_1)(2^{m-1} r_2)$ , or as  $(4^{m-1} r_1) r_2$  when  $4^{m-1} r_1 \leq 10^{300}$ . The factors are then at least  $2^{-603}$ , while  $r_1 r_2$  can be as small as  $2^{-1200}$  and would underflow. The program computes the reduced entries in increasing reduced grade and then the values  $G'_{(p,q)}$  in increasing  $p + q$ . It forms each value by adding its summands to a running sum, one at a time, in a fixed order, starting from zero. In (39) the terms with fixed  $h$  and  $c$  are first added into an inner sum over  $t$ . The programs use IEEE binary64 arithmetic with rounding to nearest and are compiled without options, such as `-ffast-math`, that allow the compiler to reassociate or otherwise reorder floating-point operations. The compiler may fuse a multiplication with the addition that follows it.

The sampler follows Proposition 8.1. Its state is the control  $(p, q)$  together with the stack of blocks, each carrying its size and the control at which it must be exposed. At an empty-stack state the candidates of a letter choice are the summands of (39), and the stored value of the state is  $G'_{(p,q)}$ . At a state whose head has size  $\ell$  and is to be exposed at control  $(c, t)$ , the stored value is  $R'_{\ell,a}(q, t)$  with  $a = p - c$ , and the candidates are the summands of (34). For  $\ell \geq 2$  the term  $D_{\ell,a}(q, t)$  is two candidates, one for each endpoint of the head. Each candidate is given its value in the scaled recurrence, so that the exact values of the candidates of a state sum to the exact scaled value of the state.

The candidates are scanned in a fixed order, in groups. Each band term and each endpoint is a group of its own. At an empty-stack state the terms of the last sum of (39) with fixed  $h$  and  $c$  form one group, and inside a head the split terms with fixed  $\ell_1$  and  $a_1$  form one group. The sampler computes the sum of each group by adding its members in order, and the running sums  $A_1 \leq A_2 \leq \dots$  of the group sums by adding these in order. It takes a multiple  $x$  of  $2^{-53}$  in  $[0, 1)$  from 53 random bits and rounds  $x\hat{T}$  to  $y$ , where  $\hat{T}$  is the stored value of the state. The random bits come from the generator xoshiro256\*\* of Blackman and Vigna [9], seeded for each sample through splitmix64. The sampler then takes the first group  $g$  with  $A_g > y$  and, inside that group, the first member whose running sum exceeds the rounded difference  $y - A_{g-1}$ , or the last member if there is none. Candidates of value zero are skipped. If no group has  $A_g > y$ , the sampler scans again with  $y$  multiplied by  $\hat{A}/\hat{T}$ , where  $\hat{A}$  is the total of the scan, and if that also falls short it takes the last candidate of positive value.

**A.2. The error bound.** We write  $\text{fl}(z)$  for  $z$  rounded to binary64,  $u = 2^{-53}$  for the unit roundoff and  $\gamma_m = mu/(1 - mu)$  for  $mu < 1$ . If  $z$  is the sum or the product of two binary64 numbers, or a product plus a number in a fused operation, and  $|z|$  lies in the normal range, then  $\text{fl}(z) = z(1 + \delta)$

with  $|\delta| \leq u$ . A multiplication by a power of two is exact when the result is normal. If  $|\theta_j| \leq \gamma_j$  and  $|\theta_k| \leq \gamma_k$ , then  $(1 + \theta_j)(1 + \theta_k) = 1 + \theta$  with  $|\theta| \leq \gamma_{j+k}$ , and a product of  $m$  factors  $1 + \delta_i$  with  $|\delta_i| \leq u$  is  $1 + \theta$  with  $|\theta| \leq \gamma_m$  [17, Lemmas 3.1 and 3.3].

For  $R_{\ell,a}(q, s)$  put  $\lambda = \ell + a + q - s$ , and for  $G_{(p,q)}$  put  $\lambda = p + q$ . This is the number of letters read by the stopped paths of  $R_{\ell,a}(q, s)$  and by the complete paths from the state  $((p, q), \emptyset)$ , and  $\lambda \leq N$ . Every band term and every  $D$  term has one letter fewer. The two factors of a split term together have  $\lambda - 1$  letters, since the split reads one letter itself, and the same holds for the two factors of a term of the last sum of (39). Put  $\nu = 3\,352\,801$ . We count the summands of (39), and of (34) for fixed  $s$ , with every index in the support of Lemma 7.1, counting  $D_{\ell,a}$  with  $\ell \geq 2$  twice, once for each endpoint. For  $G_{(p,q)}$  with  $n = p + q$  there are  $n$  of them if  $q = 0$ , and  $n + (n - 2)(p + 1)(q - 1)/2 \leq N + N^2(N - 2)/8 = \nu - 1$  if  $q \geq 1$ , since  $(p + 1) + (q - 1) = n$ , with equality at (149, 151). For  $R_{\ell,a}(q, s)$  with  $t = a + q$  there are at most  $t + 1$  if  $\ell = 1$ , and at most  $t + \ell + (\ell - 2)t(t + 1)/2 \leq N + N(N - 2)^2/8 < \nu - 1$  if  $\ell \geq 2$ , since the band and  $D$  terms number at most  $t + 2$ , the split terms with fixed  $\ell_1$  at most  $(a + 1)q + a(a + 1)/2 + 1 \leq t(t + 1)/2 + 1$ , while  $t + \ell \leq N$  and  $(\ell - 2)t \leq (N - 2)^2/4$ . So no value has more than  $\nu - 1$  summands and no letter choice has more than  $\nu - 1$  candidates.

**Lemma A.1** (Accuracy of the tables). *Every stored value is its scaled exact value times  $1 + \theta$  with  $|\theta| \leq \gamma_{\nu\lambda}$ . In particular, a stored value is zero if and only if its exact value is zero.*

*Proof.* We argue by induction on  $\lambda$ , in the order in which the values are computed. A nonzero exact scaled value is a positive integer times  $2^{-2w}$ , where  $w \leq N$  is the reduced grade or  $p + q$ , so it is at least  $2^{-600}$ , and so is every nonzero summand. Since the summands are nonnegative and rounding is monotone, every running sum lies between its first nonzero summand and the stored value. By the induction hypothesis, a computed product of stored factors is the exact value of its summand times a number between  $1/2$  and  $2$ , so it is at least  $2^{-601}$ . The build for  $N = 300$  checks the range of the stored values given in Section A.1, and with it and the form of the products there, every nonzero number formed lies between  $2^{-603}$  and  $2^{997}$ , so no underflow or overflow occurs.

A summand that is one stored value times a power of two is formed without rounding. A product of two stored values is formed with one rounding, or with none if it is fused with the next addition. By the induction hypothesis and since the letters of the factors add up to at most  $\lambda - 1$ , the exact product of the stored factors of a summand, with its power of two, is the exact value of the summand times  $1 + \theta$  with  $|\theta| \leq \gamma_{\nu(\lambda-1)}$ . The summand then undergoes at most  $\nu$  roundings: at most one in the product, at most one in each addition to the running sum of its group, and at most one in each addition of a group sum. Therefore the computed value is  $\sum_i z_i(1 + \theta_i)$ , where the  $z_i \geq 0$  are the exact summands and  $|\theta_i| \leq \gamma_{\nu\lambda}$ . This is  $(1 + \theta) \sum_i z_i$  with  $|\theta| \leq \gamma_{\nu\lambda}$ .  $\square$

**Proposition A.2** (Floating-point sampling). *Let  $n \leq 300$  and  $\bar{\gamma} = \gamma_{\nu N}$ . Assume that the random bits used by the sampler are independent and uniform. Then the output of the floating-point sampler is within total variation distance*

$$n \left( \frac{\bar{\gamma}}{1 - \bar{\gamma}} + 7\nu u \right) < 3.5 \cdot 10^{-5}$$

*of the uniform distribution on  $\text{Av}_n(12453)$ .*

*Proof.* By Lemma A.1 the sampler reaches only states of positive count. Fix such a state, with stored value  $\hat{T}$  and candidates  $1, \dots, k$  in scan order. Let  $w_i$  be the scaled exact value of candidate  $i$  and  $W = w_1 + \dots + w_k$ . Let  $v_i$  be the exact product of the stored values in candidate  $i$  and its power of two, and let  $\hat{w}_i = \text{fl}(v_i)$  be the value the sampler computes, with  $\hat{w}_i = v_i$  when there is one factor.

*The values.* By [Lemma A.1](#) applied to the factors,  $\hat{w}_i = w_i(1 + \theta_i)$  with  $|\theta_i| \leq \gamma_{\nu(\lambda-1)+1} \leq \bar{\gamma}$ , and  $\hat{T} = W(1 + \theta)$  with  $|\theta| \leq \bar{\gamma}$ . Therefore

$$\sum_{i=1}^k \left| \frac{\hat{w}_i}{\hat{T}} - \frac{w_i}{W} \right| = \sum_{i=1}^k \frac{w_i}{W} \left| \frac{1 + \theta_i}{1 + \theta} - 1 \right| \leq \frac{2\bar{\gamma}}{1 - \bar{\gamma}}. \quad (40)$$

The stored value  $\hat{T}$  and the total  $\hat{A}$  of the scan are both computed sums of  $v_1, \dots, v_k$  in which each  $v_i$  undergoes at most  $\nu$  roundings. The two endpoint candidates are one summand of the table, of value  $v_i + v_{i+1}$ . Therefore  $\hat{T}$  and  $\hat{A}$  are both within a factor  $1 \pm \gamma_\nu$  of  $v_1 + \dots + v_k$ , and  $|\hat{A} - \hat{T}| \leq 2.01\nu u \hat{T}$ .

*The scan.* For  $y < \hat{A}$  the chosen candidate is a nondecreasing function of  $y$ , since rounding is monotone. So candidate  $i$  is chosen for  $y$  in an interval  $[\xi_{i-1}, \xi_i)$ , where  $0 = \xi_0 \leq \dots \leq \xi_k = \hat{A}$ . Consider a group with running sums  $\alpha_1 \leq \dots \leq \alpha_r$  of its members, and let  $A_{g-1}$  and  $A_g$  be the running sums before and after it. The group occupies  $[A_{g-1}, A_g)$ , and its member  $j < r$  ends where  $\text{fl}(y - A_{g-1})$  reaches  $\alpha_j$ , or at  $A_g$  if that comes first. Since  $|\text{fl}(z) - z| \leq u|z|$ , this breakpoint differs from  $A_{g-1} + \alpha_j$  by at most  $1.01u\hat{A}$ . The difference  $\alpha_j - \alpha_{j-1}$  differs from the value of member  $j$  by at most  $u\hat{A}$ , and  $A_g - A_{g-1}$  differs from  $\alpha_r$  by at most  $u\hat{A}$ . Therefore

$$|\xi_i - \xi_{i-1} - \hat{w}_i| \leq 4.02u\hat{A} \quad (1 \leq i \leq k). \quad (41)$$

*The draw.* For  $\tau \geq 0$  let  $F(\tau)$  be the probability that  $y \geq \tau$ , and let  $f(\tau) = \max(0, 1 - \tau/\hat{T})$ . Since  $x\hat{T} < \hat{T}$ , we have  $y \leq \hat{T}$ , so  $F(\tau) = 0 = f(\tau)$  for  $\tau > \hat{T}$ . For  $\tau \leq \hat{T}$ , we have  $y \geq \tau$  if  $x\hat{T} \geq \tau/(1 - u)$  and only if  $x\hat{T} \geq \tau/(1 + u)$ . Since  $x$  is uniform on the multiples of  $2^{-53}$  in  $[0, 1)$ , the probability that  $x \geq z$  lies between  $1 - z - u$  and  $1 - z$ . Together,

$$|F(\tau) - f(\tau)| \leq 2.01u \quad (0 \leq \tau \leq \hat{A}). \quad (42)$$

*One letter choice.* Let  $\hat{p}_i$  be the probability that the sampler chooses candidate  $i$ . Without a rescan, candidate  $i$  is chosen with probability  $F(\xi_{i-1}) - F(\xi_i)$ . By (42), replacing  $F$  by  $f$  changes the sum over  $i$  of these probabilities by at most  $4.02ku$  in total. The differences  $f(\xi_{i-1}) - f(\xi_i)$  equal  $(\xi_i - \xi_{i-1})/\hat{T}$  except above  $\hat{T}$ , which changes the sum by at most  $\max(\hat{A} - \hat{T}, 0)/\hat{T} \leq 2.01\nu u$ . By (41), replacing  $\xi_i - \xi_{i-1}$  by  $\hat{w}_i$  changes it by at most  $4.03ku$ . A rescan happens with probability  $F(\hat{A}) \leq f(\hat{A}) + 2.01u \leq 2.01\nu u + 2.01u$ . Since  $k < \nu$ ,

$$\sum_{i=1}^k \left| \hat{p}_i - \frac{\hat{w}_i}{\hat{T}} \right| \leq 8.05ku + 4.02\nu u + 2.01u \leq 14\nu u.$$

With (40), the total variation distance between the choice of the sampler and the choice with probabilities  $w_i/W$  is at most  $\bar{\gamma}/(1 - \bar{\gamma}) + 7\nu u$ .

*The sample.* With probabilities  $w_i/W$ , the choices produce the uniform distribution on  $\text{Av}_n(12453)$ , by [Proposition 8.1](#). Run the two processes together, and as long as they are in the same state, couple their next choices so that they differ with probability equal to the total variation distance between them. A sample is produced by  $n$  choices, so the two outputs differ with probability at most  $n(\bar{\gamma}/(1 - \bar{\gamma}) + 7\nu u)$ . At  $n = 300$  this is  $3.43 \cdot 10^{-5}$ .  $\square$

The bound of [Lemma A.1](#) is far from the observed error: at  $n = 150$  it gives about  $6 \cdot 10^{-8}$  for  $G_{(n,0)}$ , while the floating-point values  $G_{(n,0)}$  agree with the exact coefficients through  $n = 150$  to within  $3.2 \cdot 10^{-13}$  relative error.

## APPENDIX B. THE LEAN DEVELOPMENT

This appendix describes the Lean development of [Section 9](#).

**B.1. Conventions.** The development works with words over  $\mathbb{N}$  and 0-based values. It defines pattern containment as the existence of an order-isomorphic subsequence, a finite set of 0-based words in bijection with  $\text{Av}_n(\tau)$ , the one-threshold recurrence  $W$  of (5) and (6), the literal recurrence  $H$  of (20) with  $H_0(\emptyset) = 1$  for  $d = 2$ , and the kernel recurrences (12)–(13) for  $d = 1$  and (25)–(26) for  $d = 2$ . In the two kernel systems every sum over intermediate or terminal controls is restricted to the controls of mass at most that of the source control of its kernel row (for  $d = 1$ , to  $u \leq p$  in (12) and to  $t \leq h$  in (13)), which Lemmas 2.17 and 6.1 show loses nothing. In `Challenge.lean` the four main theorems of Section 9 are named `Av12453.count_1342_literal`, `Av12453.count_1342_kernel`, `Av12453.count_12453_literal`, and `Av12453.count_12453_kernel`. They restate for Mathlib’s permutations the development’s theorems `av1342_count`, `av1342_count_kernel`, `av12453_count`, and `av12453_count_kernel`, which are stated for words.

**B.2. Organization.** The Lean project `formal/Av12453/` builds two libraries from one source tree. The library `PermPatterns/` holds the generic apparatus (words, containment and avoidance, standardization, the patterns 231 and  $\iota_d$ , sums, symmetries, and the passage to Mathlib’s permutations of  $\{0, \dots, n-1\}$ ), and nothing in it refers to the family  $\beta_d$ . The library `Av12453/` holds the development specific to this paper. The file `Av12453/Axioms.lean`, which neither library imports, checks that every declaration of the two libraries depends only on propositional extensionality, choice, and quotient soundness. The file `formal/README.md` describes the modules, the checks run by `check.sh`, and the trusted base. The Lean toolchain (version 4.35.0-rc2) and the Mathlib revision are pinned in the project files.

**B.3. What must be read.** The file `Challenge.lean` states the four theorems for Mathlib’s permutations of  $\{0, \dots, n-1\}$ , with pattern containment `PermContains` and the patterns written as products of transpositions, together with the definitions of `W`, `H`, `K`, and `G`, and `comparator.sh` checks that these definitions are identical to those of the development. The recurrences are written with a recursion bound. The definitions `W`, `H`, `K`, and `G` evaluate an auxiliary function at a bound exceeding, respectively, the number of unread values, the number of unread values, the grade, and the control mass of the argument, and the development proves that every sufficient bound gives the same value. So the reader need only compare the equations of the auxiliary functions with the displays (5) and (6), (20) with  $H_0(\emptyset) = 1$ , (12)–(13), and (25)–(26). `Challenge.lean` writes the terms with  $a = 0$  or  $b = 0$  of the split sums of these displays as one endpoint term, with a case distinction on whether the head has size one, which is the same sum. The development’s restated kernel equations call this term `D`, and those of `W` and `H` call it `Eend`. The opening docstring of `Challenge.lean` gives this dictionary, with the 0-based conventions spelled out, and the displays are also restated as proved theorems (`K_succ_eq`, `G_eq`, and their analogues), which serve as reading aids but are not part of what must be trusted.

**B.4. Proofs and checks.** The thresholds  $b_1, b_2$  are defined as the least final values of increasing subsequences, and the interval stack by the moves of Proposition 4.1(b), which classify a letter by the band or interval containing it, so Lemma 2.7 and Proposition 2.8(a) and (b) are proved for this notion of legal prefix. Lemmas 2.1 and 2.2 are formalized, but the four main theorems do not use them. Since the kernels are defined by their recurrences, Proposition 5.2 is not needed. The restriction of the control sums is proved lossless, and it is also what makes recursion on the grade terminate for (12) and (25) read as definitions.

The development has over eleven thousand lines in thirty modules. The definitions were also exercised by kernel evaluation: the scans of the running example (4) reproduce the rows of Examples 2.9 and 4.2, kernel evaluation of the recurrences reproduces the first seven terms of (2), and compiled evaluation the first ten.

## REFERENCES

1. Michael H. Albert, Christian Bean, Anders Claesson, Émile Nadeau, Jay Pantone, and Henning Ulfarsson, *Combinatorial exploration: an algorithmic framework for enumeration*, Mem. Amer. Math. Soc. **317** (2026), no. 1611, 109 pp.
2. ———, *The Permutation Pattern Avoidance Library (PermPAL)*, <https://permpal.com>, 2026, Accessed September 6, 2026.
3. Michael H. Albert, Cheyne Homberger, Jay Pantone, Nathaniel Shar, and Vincent Vatter, *Generating permutations with restricted containers*, J. Combin. Theory Ser. A **157** (2018), 205–232.
4. David Aldous and Persi Diaconis, *Longest increasing subsequences: from patience sorting to the Baik–Deift–Johansson theorem*, Bull. Amer. Math. Soc. (N.S.) **36** (1999), no. 4, 413–432.
5. Ragnar Pall Ardal, Arnar Bjarni Arnarson, Christian Bean, Alfur Birkir Bjarnason, Jon Steinn Eliasson, Bjarki Agust Gudmundsson, Sigurjón Ingi Jónsson, Bjarni Jens Kristinsson, Tomas Ken Magnusson, Émile Nadeau, Jay Pantone, Murray Tannock, and Henning Ulfarsson, *Permuta: a Python library for permutations and patterns*, <https://github.com/PermutaTriangle/Permuta>, 2025, Version 2.3.1, released June 13, 2025, accessed September 14, 2026.
6. Richard Arratia, *On the Stanley–Wilf conjecture for the number of permutations avoiding a given pattern*, Electron. J. Combin. **6** (1999), Note 1, 4 pp.
7. Jörgen Backelin, Julian West, and Guoce Xin, *Wilf-equivalence for singleton classes*, Adv. in Appl. Math. **38** (2007), no. 2, 133–148.
8. Yonah Biers-Ariel, *Julia program to compute terms of OEIS A116485*, [https://oeis.org/A116485/a116485\\_1.txt](https://oeis.org/A116485/a116485_1.txt), 2019, Accessed August 28, 2026.
9. David Blackman and Sebastiano Vigna, *Scrambled linear pseudorandom number generators*, ACM Trans. Math. Software **47** (2021), no. 4, 1–32.
10. Miklós Bóna, *Exact enumeration of 1342-avoiding permutations: a close link with labeled trees and planar maps*, J. Combin. Theory Ser. A **80** (1997), no. 2, 257–272.
11. ———, *The limit of a Stanley–Wilf sequence is not always rational, and layered patterns beat monotone patterns*, J. Combin. Theory Ser. A **110** (2005), no. 2, 223–235.
12. Noam Chomsky and Marcel P. Schützenberger, *The algebraic theory of context-free languages*, Computer Programming and Formal Systems (Paul Braffort and David Hirschberg, eds.), Studies in Logic and the Foundations of Mathematics, vol. 35, North-Holland, Amsterdam, 1963, pp. 118–161.
13. Nathan Clisby, Andrew R. Conway, Anthony J. Guttmann, and Yuma Inoue, *Classical length-5 pattern-avoiding permutations*, Electron. J. Combin. **29** (2022), no. 3, Paper No. P3.14, 63 pp.
14. Leonardo de Moura and Sebastian Ullrich, *The Lean 4 theorem prover and programming language*, Automated Deduction – CADE 28, Lecture Notes in Computer Science, vol. 12699, Springer, 2021, pp. 625–635.
15. Philippe Flajolet and Robert Sedgewick, *Analytic combinatorics*, Cambridge University Press, Cambridge, 2009.
16. Scott Garrabrant and Igor Pak, *Permutation patterns are hard to count*, Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms (Arlington, VA), SIAM, 2016, pp. 923–936.
17. Nicholas J. Higham, *Accuracy and stability of numerical algorithms*, second ed., Society for Industrial and Applied Mathematics, Philadelphia, PA, 2002.
18. John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman, *Introduction to automata theory, languages, and computation*, second ed., Addison-Wesley, Boston, MA, 2001.
19. Adam Marcus and Gábor Tardos, *Excluded permutation matrices and the Stanley–Wilf conjecture*, J. Combin. Theory Ser. A **107** (2004), no. 1, 153–160.
20. Albert Nijenhuis and Herbert S. Wilf, *Combinatorial algorithms for computers and calculators*, second ed., Academic Press, New York, 1978.
21. OEIS Foundation Inc., *Sequence A116485: Number of permutations in  $S_n$  that avoid the pattern 12453 (or equivalently, 31245)*, The On-Line Encyclopedia of Integer Sequences, <https://oeis.org/A116485>, 2026, Accessed August 28, 2026.
22. Thomas Reps, Stefan Schwoon, Somesh Jha, and David Melski, *Weighted pushdown systems and their application to interprocedural dataflow analysis*, Sci. Comput. Program. **58** (2005), no. 1–2, 206–263.
23. Michael Sipser, *Introduction to the theory of computation*, third ed., Cengage Learning, Boston, MA, 2013.
24. The mathlib Community, *The Lean mathematical library*, Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020), ACM, 2020, pp. 367–381.
25. Herbert S. Wilf, *What is an answer?*, Amer. Math. Monthly **89** (1982), no. 5, 289–292.
26. Doron Zeilberger, *Enumeration schemes and, more importantly, their automatic generation*, Ann. Comb. **2** (1998), no. 2, 185–195.

DEPARTMENT OF COMPUTER SCIENCE, REYKJAVIK UNIVERSITY, MENNTAVEGUR 1, 102 REYKJAVIK, ICELAND  
*Email address:* **henningu@ru.is**